

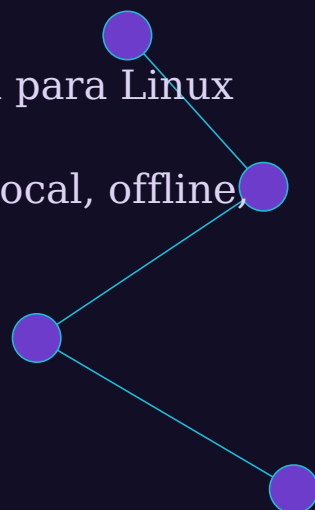
PROJETO ODISSEIA | ATHENAS OS

ATENA

Engenharia de uma Inteligência Artificial Nativa para Linux

Construindo um agente de sistema operacional local, offline, híbrido e extensível para o Athenas OS

Livro técnico, didático e de engenharia
Edição de projeto 0.1 - agosto de 2026
Debian 13 | amd64 | ARM64 | ARMv7 | RISC-V



Nota editorial e estado do projeto

Este livro descreve como construir a plataforma Atena e integra-la ao Athenas OS. Ele não afirma que os componentes propostos já estejam implementados. Toda afirmação de estado é rastreada como requisito do Projeto Odisseia, decisão da arquitetura técnica, recurso observado no script atual, inspiração do Deepin/UOS AI ou nova proposta desta obra.

O objetivo pedagógico é deliberadamente ambicioso: conduzir uma pessoa com alguma experiência em Python, mas sem domínio de C, até a compreensão, implementação, compilação, teste, empacotamento e integração de um agente nativo de sistema operacional. Os exemplos priorizam contratos simples, privilégio mínimo, diagnóstico e degradação graciosa em hardware limitado.

Sumario

PARTE I - O QUE ESTAMOS CONSTRUINDO	7
1. A definição rigorosa da Atena	7
2. O lugar da Atena no Projeto Odisseia	8
3. Rastreabilidade: fato, estado, inspiração e proposta	8
4. Estudo de caso: Deepin e UOS AI	9
5. Princípios de produto transformados em requisitos	10
6. A arquitetura final em camadas	11
7. Quatro modos operacionais	12
8. Estrutura de componentes e processos	13
9. A Atena como mentora socrática	14
10. Critérios de sucesso e antiobjetivos	15
PARTE II - FUNDAMENTOS DE INTELIGÊNCIA ARTIFICIAL	16
11. IA, aprendizado de máquina e modelos generativos	16
12. A matemática mínima, construída por sentido	16
13. Como uma rede neural aprende	17
14. Tokens, embeddings e Transformers	18
15. Quantização, contexto e custo real de inferência	18
16. Usar, adaptar ou treinar	19
17. Mini Transformer educacional em Python	19
18. Avaliação de modelos e do sistema completo	21
PARTE III - C DO ZERO, DENTRO DA ATENA	22
19. Código-fonte, compilador e programa	22
20. Tipos, variáveis e representação	22
21. Decisão e repetição	23
22. Funções, contratos e erros	24
23. Arrays, strings e limites	24
24. Structs, enums e modelagem de domínio	25
25. Ponteiros sem misticismo	26

26. Stack, heap, malloc e ciclo de vida	26
27. Ponteiros de função e interfaces substituíveis	27
28. Headers, módulos e encapsulamento	28
29. CMake e build reproduzível	28
30. Arquivos, JSON, SQLite e HTTP	29
31. Processos, threads, sinais e IPC	30
32. Debugging, sanitizers e testes em C	30
33. Linux nativo: XDG, systemd, D-Bus e Polkit	31
34. Segredos e higiene operacional	32

PARTE IV - CONSTRUINDO O ATENA CORE 33

35. Desenvolvimento vertical e contratos primeiro	33
36. Request Engine e Context Builder	33
37. Sessão, histórico, memória e conhecimento	34
38. Provider API e descoberta de capacidades	35
39. Provider Router e Model Router	36
40. Privacy Engine	37
41. Policy Engine e decisões de autorização	37
42. Tool Engine: efeitos tipados, não texto de shell	38
43. Agent Engine	39
44. Skill API e Skill Manager	39
45. Auditoria e observabilidade	40
46. Configuração, schema e migrações	41

PARTE V - HARDWARE, MODELOS E PROVIDERS 42

47. Hardware Profiler	42
48. Compatibility Engine	42
49. Benchmark Engine	43
50. Model Manager	44
51. llama.cpp: runtime local de referência	45
52. Fase A: Core via llama-server	46

53. Fase B: integração com libllama	46
54. Ollama como provider local opcional	47
55. Providers em nuvem e OpenAI-compatible	48
56. Custo, rate limits e privacidade operacional	48
57. Perfis por arquitetura	49

PARTE VI - CONHECIMENTO, BUSCA, VOZ E EXPERIÊNCIA 50

58. Search Provider independente	50
59. RAG: recuperação antes de geração	51
60. RAG Lite com SQLite FTS5	51
61. RAG Standard e Advanced	52
62. Citation Engine	53
63. Personal Knowledge Base	53
64. Assistente de voz opcional	53
65. Atena CLI	54
66. Atena UI, sidebar e overlay	55
67. Model, Provider, Agent e Skill Managers na interface	55
68. Experiência pedagógica e transparência	56
69. ATENA SECURITY LAB	56

PARTE VII - EMPACOTAR, INTEGRAR, TESTAR E LANÇAR 58

70. Pacotes Debian da Atena	58
71. Serviços, D-Bus, Polkit e desktop no pacote	59
72. Integrando à build do Athenas OS	59
73. Update Engine e compatibilidade de dados	60
74. Estratégia completa de testes	61
75. Testes de segurança ofensiva autorizada da própria Atena	61
76. Evals pedagógicas e de confiabilidade	62
77. CI, release e cadeia de suprimentos	62
78. Roadmap da Atena	63
79. Projeto final: uma Atena mínima utilizável	64

PARTE VIII - IMPLEMENTAÇÃO DE REFERÊNCIA E CONTRATOS 65

80. Esqueleto de diretórios compilável	65
81. Provider mock	65
82. Router determinístico inicial	67
83. Registro de tools	67
84. Migração SQLite	68
85. Contrato de Skill completo	68
86. Checklist de Definition of Done	68

PARTE IX - REFERÊNCIAS, GLOSSÁRIO E PRÓXIMOS PASSOS 70

87. Glossário essencial	70
88. Fontes primárias do Projeto Odisseia	70
89. Referências oficiais de integração com IA no sistema	71
90. Referências oficiais da pilha técnica	71
91. Referências científicas fundamentais	71
92. Referências de APIs de providers	71
93. Decisões de engenharia que devem virar ADRs	72
94. Próximo passo concreto	72
95. Conclusão	72

PARTE I - O QUE ESTAMOS CONSTRUINDO

1. A definição rigorosa da Atena

A Atena é a plataforma de inteligência artificial do Athenas OS. Essa frase parece simples, mas impede o erro arquitetural mais perigoso do projeto: confundir o produto com o modelo que responde ao usuário. Um modelo de linguagem calcula probabilidades e gera tokens. A Atena organiza identidade, contexto, memória, conhecimento, ferramentas, permissões, interfaces, modelos e provedores para produzir uma experiência coerente de sistema operacional.

Em termos de engenharia, o modelo é um componente substituível. Hoje a máquina pode usar um arquivo GGUF por meio do `llama.cpp`; amanhã pode usar Ollama, um endpoint compatível com a API da OpenAI, um serviço corporativo ou outro runtime ainda inexistente. A troca do motor cognitivo não deve apagar a personalidade pedagógica, o histórico autorizado, as políticas de privacidade, as Skills instaladas, o catálogo de ferramentas nem a integração com o Linux.

Portanto, as seguintes equivalências são falsas:

- Atena não é um LLM.
- Atena não é `llama.cpp`.
- Atena não é Ollama.
- Atena não é uma janela de chatbot.
- Atena não é um shell com linguagem natural.
- Atena não é uma API comercial renomeada.

A definição operacional correta é: **Atena é um conjunto de serviços e interfaces que recebe uma intenção humana, reúne contexto permitido, escolhe capacidades adequadas, consulta um ou mais motores cognitivos, executa somente ferramentas autorizadas e devolve uma resposta rastreável.**

FONTE DO PROJETO Requisito pedagógico original

O Projeto Odisseia descreve a Atena como socrática, híbrida e regional. Essas propriedades pertencem à plataforma e ao seu método, não aos pesos de um modelo específico.

PROPOSTA DO LIVRO Contrato de identidade

A personalidade, o método socrático, as políticas e o contexto regional devem residir em arquivos e serviços versionados do projeto. O prompt de um provider é uma projeção temporária desse estado, nunca sua única cópia.

O produto em uma frase

Se o usuário remover todos os modelos, a Atena ainda deve conseguir abrir, mostrar configurações, gerenciar a base de conhecimento, explicar por que não pode responder e permitir a instalação de um novo motor. Se o usuário trocar de provider, a experiência permanece reconhecível. Se a rede cair, o sistema não desaparece. Esse comportamento distingue uma plataforma de um simples cliente de API.

2. O lugar da Atena no Projeto Odisseia

O Projeto Odisseia é um ecossistema educacional. O Cyberdeck Odisseia é a família de dispositivos físicos. O Athenas OS é a distribuição Linux e a plataforma de software. Atena é a camada de inteligência e mediação pedagógica. *OdysseyOS* permanece um nome histórico dos materiais iniciais e não deve coexistir publicamente como se fosse sinônimo técnico de Athenas OS.

Nome	Domínio	Responsabilidade
Projeto Odisseia	Ecossistema	Missão educacional, hardware, software e comunidade
Cyberdeck Odisseia	Dispositivo	Computador modular, reparável e adaptado ao propósito
Athenas OS	Distribuição Linux	Base Debian, desktop, pacotes, atualizações e integração
Atena	Plataforma de IA	Mentoria, agentes, modelos, conhecimento, ferramentas e políticas
OdysseyOS	Nome histórico	Referência conceitual que deve ser resolvida antes de um release público

Os materiais visuais do projeto associam o cyberdeck a autonomia, cultura maker, reaproveitamento, identidade própria e compreensão do computador por dentro. Essa filosofia influencia a Atena diretamente. Uma IA educacional coerente com o Odisseia não deve esconder cada ação atrás de uma animação. Ela deve poder explicar qual provider foi escolhido, por que uma ferramenta pediu confirmação, quais fontes sustentam uma resposta e quanto o modelo consumiu da máquina.

O objetivo não é romantizar hardware antigo. Uma máquina com 2 GB de RAM não se torna adequada a um LLM de bilhões de parâmetros por causa de uma boa intenção. O objetivo é degradar com honestidade: oferecer busca lexical, documentação, diagnósticos e automações tipadas quando inferência local pesada não for viável; permitir provider remoto somente com consentimento; e preservar o funcionamento básico do Athenas OS sem IA.

A pergunta do produto não é “qual modelo vamos instalar?”, mas “qual experiência continua útil, segura e explicável em cada classe de hardware?”.

O que este livro inclui e exclui

Este livro inclui tudo o que for necessário para compilar, instalar, executar, empacotar, atualizar e integrar a Atena. Isso exige tratar de Debian, systemd, D-Bus, Polkit, PipeWire, pacotes .deb e inclusão na ISO. Ele não ensina a construir uma distribuição Linux completa, desenhar um bootloader ou reproduzir toda a configuração do desktop. Esses assuntos aparecem apenas nas fronteiras em que a IA encontra o sistema.

3. Rastreabilidade: fato, estado, inspiração e proposta

Um documento de engenharia perde valor quando transforma mockups em recursos implementados. Para evitar isso, esta obra utiliza cinco classes de afirmação.

Marcador	Significado	Exemplo
FONTE DO PROJETO	Requisito presente nos materiais originais	Atena socrática, híbrida e regional

Marcador	Significado	Exemplo
DECISÃO ANTERIOR	Escolha já registrada na arquitetura técnica	Core desacoplado, Model Router e providers
ESTADO DO SCRIPT	Recurso verificável no construtor atual	Bookworm, amd64, ISO Live, XFCE/GNOME/KDE e branding
INSPIRAÇÃO UOS AI	Experiência pública estudada, sem cópia proprietária	seleção de texto, taskbar de IA e Agent Store
PROPOSTA DO LIVRO	Nova decisão especificada aqui	contratos em C, score de compatibilidade e protocolo de Skills

ESTADO DO SCRIPT O que existe hoje

O arquivo `criar-athenas-linux-corrigido.sh` produz uma configuração Debian Live Bookworm para amd64, escolhe XFCE, GNOME ou KDE, aplica identidade visual, evita executar o script inteiro como root, valida arquivos essenciais e preserva os caminhos de kernel e initramfs gerados pelo live-build. A Atena ainda não faz parte da build principal.

DECISÃO ANTERIOR Direção técnica já registrada

A arquitetura técnica de agosto de 2026 recomenda Debian 13 Trixie, XFCE como referência leve, pacotes próprios, lançamento inicial em amd64, evolução para ARM64 e canal experimental RISC-V. Ela posiciona o marco Atena depois da base do sistema e do repositório de pacotes.

O contraste Bookworm versus Trixie não deve ser apagado. O script representa um estágio testável; a arquitetura representa a próxima base. O trabalho de integração da Atena deve aceitar ambos durante a transição, mas o empacotamento novo deve ser validado prioritariamente no Debian 13.

4. Estudo de caso: Deepin e UOS AI

O Deepin/UOS AI é a principal referência pública de experiência porque trata IA como capacidade transversal do sistema operacional, não apenas como aplicativo isolado. A versão UOS AI 2.0 apresentou três entradas sistêmicas: AI Search, AI FollowAlong e AI Taskbar. O Deepin 25 consolidou um ponto central para modelos, agentes e configurações, seleção de texto com ações, busca em linguagem natural, arrastar arquivos e Agent Store. O Deepin 25.1 acrescentou Claw Mode, Skills Center, sidebar entre aplicações e roteamento de modelos por tarefa.

Essas referências respondem a uma pergunta de produto: como a IA aparece no momento em que o usuário precisa dela? A Atena deve estudar os padrões de interação e construir contratos próprios, abertos e auditáveis.

Experiência pública	Intenção de uso	Tradução aberta para Atena
AI FollowAlong	Agir sobre texto selecionado	serviço <code>atena-selection</code> , menu extensível e Skills tipadas
AI Taskbar	Soltar arquivo e receber ações	alvo de arrastar com análise local de MIME e permissões
AI Sidebar	Manter IA ao lado de outro app	janela lateral independente do provider
AI Search	Buscar por linguagem natural, inclusive offline	Search Provider + FTS5 + embeddings opcionais
Agent Store	Instalar agentes especializados	manifesto aberto, assinatura e Skill Manager
Skills Center	Criar e importar capacidades	Skill API versionada, schema e auditoria
Claw Mode	Controlar tarefas do computador por linguagem natural	agente com ferramentas tipadas, política e confirmações

Experiência pública	Intenção de uso	Tradução aberta para Atena
Model Configuration	Alternar modelos locais e remotos	Provider Manager + Model Manager + capability discovery

INSPIRACAO UOS AI
O que pode ser aprendido

O valor do UOS AI está na integração contextual: texto selecionado, arquivos arrastados, busca, sidebar, taskbar e configurações unificadas. A Atena adota o propósito dessas experiências, não código fechado, identidade visual, endpoints privados nem decisões que contrariem o hardware e a segurança do Odisseia.

O limite mais importante da inspiração

O Deepin 25.1 anuncia controle automático do computador por linguagem natural. Para a Atena, isso não autoriza um canal oculto de root nem execução direta de texto gerado pelo modelo. A equivalência de propósito deve ser implementada como intenção estruturada, validação de schema, decisão de política, confirmação humana e ferramenta conhecida. Quanto maior a autonomia percebida, mais rígida precisa ser a fronteira técnica.

5. Princípios de produto transformados em requisitos

Princípios só ajudam a engenharia quando produzem critérios verificáveis.

Local-first e offline-capable

Local-first significa que o caminho local é tratado como cidadão de primeira classe. Offline-capable significa que funções essenciais continuam disponíveis sem rede. Isso não obriga todo hardware a executar um LLM; obriga a Atena a oferecer um modo offline honesto, com documentação, RAG lexical, ferramentas locais e UI funcional.

Cloud optional e provider-agnostic

Nenhuma API comercial pode ser requisito de instalação. O contrato de provider deve aceitar implementações locais, remotas, corporativas e compatíveis com padrões conhecidos. Remover um provider não pode corromper conversas, Skills ou memória.

Model-agnostic

Modelos anunciam capacidades, limites de contexto, formato, licença e requisitos. O Core solicita uma capacidade; o Model Router escolhe um modelo adequado dentro do provider selecionado. Nomes comerciais não aparecem espalhados pela lógica central.

Hardware-aware

Atena mede CPU, ISA, RAM disponível, swap, zram, GPU, VRAM, backend, armazenamento, energia e temperatura quando disponível. Compatibilidade começa com estimativa e passa a usar benchmark real da própria máquina.

Privacy-aware e permission-aware

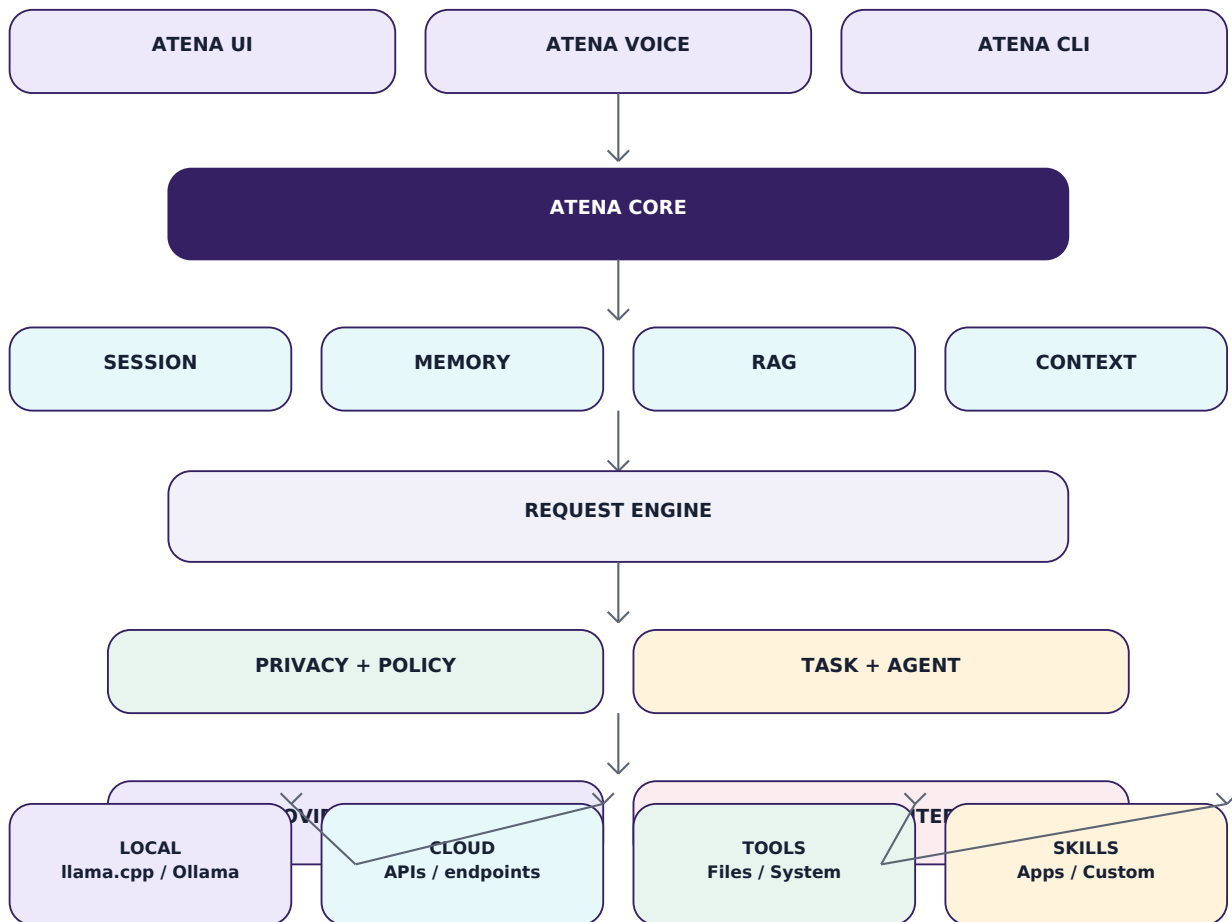
Conteúdo privado não cruza a rede silenciosamente. Ferramentas não recebem privilégios implícitos. Arquivos, diretórios e tipos de dado podem ter políticas próprias. Toda elevação administrativa é realizada por componente pequeno, identificável e auditável.

Modular e multi-arquitetura

Cada subsistema possui contrato estável, testes e ciclo de versão. Código C/C++ é compilado por arquitetura. Schemas, prompts e assets independentes da arquitetura são empacotados como `Architecture: all`. O lançamento de uma arquitetura depende de testes reais, não apenas de uma compilação bem-sucedida.

6. A arquitetura final em camadas

Arquitetura de referencia da plataforma Atena



A arquitetura começa por três interfaces principais: UI, Voice e CLI. Todas chamam o mesmo Atena Core. Essa regra impede que a janela gráfica tenha uma lógica de privacidade diferente do terminal ou que a voz use ferramentas não disponíveis ao texto.

O Core coordena sessão, memória, RAG e contexto. O Request Engine normaliza a entrada e produz um objeto interno imutável. Privacy Engine e Policy Engine classificam os dados e as ações. Task Engine e Agent Engine decidem se a solicitação é uma resposta direta, uma recuperação de conhecimento ou um fluxo de ferramentas. Provider Router escolhe o tipo de execução cognitiva; Tool Router controla efeitos no sistema.

Componentes de gestão ficam fora do caminho crítico de cada token, mas usam os mesmos contratos:

- Model Manager instala, verifica, registra, testa e remove modelos.

- Provider Manager configura endpoints e segredos por referência.
- Agent Manager organiza agentes e workflows.
- Skill Manager valida manifestos, permissões, dependências e versões.
- Hardware Profiler produz um retrato normalizado da máquina.
- Benchmark Engine substitui suposições por medições.
- Update Engine aplica migrações de configuração, banco e pacotes.

Fluxo de uma requisição

Roteamento de uma requisicao



O classificador de tarefa não precisa ser um LLM grande. Regras determinísticas resolvem comandos conhecidos; um modelo pequeno pode auxiliar intenções ambíguas; o usuário confirma ações de risco. A Privacy Policy é executada antes do envio a qualquer provider remoto. O Capability Resolver elimina providers incapazes antes de calcular qualidade, custo ou velocidade.

LIMITE DE SEGURANCA Ordem não negociável

Privacidade e autorização vêm antes do roteamento econômico ou qualitativo. Um provider remoto excelente não é candidato quando a política determina processamento local.

7. Quatro modos operacionais

Quatro modos operacionais



Offline real

Rede indisponível ou bloqueada. Atena utiliza modelo local quando viável, RAG local, FTS5, documentação e ferramentas locais. Providers remotos, pesquisa web, embeddings remotos, STT remoto e TTS remoto ficam desabilitados. A UI mostra essa condição sem sugerir que uma operação remota “falhou por acaso”.

Local conectado

O modelo permanece local, mas Search Provider e serviços autorizados podem usar rede. Prompts e documentos continuam locais sempre que possível. O fato de um modelo executar localmente não implica isolamento da internet; o acesso ocorre somente por ferramenta governada.

Cloud

O modelo principal está em endpoint remoto. A Atena ainda executa política, memória e ferramentas localmente. Antes do envio, o Core reduz contexto, remove segredos e informa ao usuário quando arquivos ou trechos serão transmitidos.

Híbrido

O router considera privacidade, capacidade, hardware, disponibilidade, latência, custo, qualidade e preferência. Uma tarefa pode fazer recuperação local, consultar busca autorizada e sintetizar com modelo local ou remoto. “Híbrido” não significa comportamento imprevisível: a decisão precisa ser explicável e reproduzível a partir da política ativa.

8. Estrutura de componentes e processos

Uma implementação nativa não exige que tudo seja um único binário. Separar processos reduz impacto de falhas e facilita privilégios mínimos.

Processo ou biblioteca	Privilegio	Papel
atena-core	usuário	sessões, roteamento, memória, RAG, agents e auditoria
atena-ui	usuário	interface gráfica e acessibilidade
atena-cli	usuário	cliente terminal e diagnóstico
atena-voice	usuário	captura, STT, interrupção e TTS
atena-modeld	usuário	catálogo, downloads e benchmarks de modelos
atena-searchd	usuário	indexação local e pesquisa autorizada
atena-tool-*	usuário ou mecanismo específico	implementação pequena de ferramenta
helper administrativo	root somente durante a ação	operações limitadas protegidas por Polkit

O primeiro protótipo pode ligar CLI e Core como biblioteca no mesmo processo. A segunda etapa move o Core para um serviço de usuário e usa socket Unix ou D-Bus. Essa progressão evita introduzir IPC antes de o contrato interno estar estável, mas mantém claro que a versão final é um serviço compartilhado por múltiplas interfaces.

Organização proposta do repositório

Diretório	Conteúdo
core/	request, sessão, roteadores, política e eventos
include/atena/	API pública em C
providers/	llama, Ollama, cloud e endpoints customizados
tools/	ferramentas tipadas e implementações
skills/	parser, validação e runtime de Skills
rag/	extração, chunking, FTS5, vetores e citação
memory/	sessão, histórico, preferências e longo prazo
voice/	PipeWire, STT, TTS e wake word opcional
ui/	aplicação gráfica e integrações de desktop
cli/	cliente e comandos de diagnóstico
packaging/debian/	pacotes, serviços, políticas e maintainer scripts
tests/	unidade, integração, segurança, golden e hardware
schemas/	requests, providers, tools, skills e auditoria
docs/	decisões arquiteturais, protocolos e manuais

9. A Atena como mentora socrática

O método socrático não é sinônimo de responder toda pergunta com outra pergunta. Em educação, a estratégia deve observar intenção, conhecimento prévio, urgência e risco. Se o estudante pede ajuda para entender um ponteiro, a Atena pode diagnosticar o modelo mental, propor uma analogia, pedir uma previsão sobre o código, executar um exemplo e corrigir a hipótese. Se o usuário pede um comando de recuperação urgente, perguntas excessivas atrapalham.

O comportamento pedagógico deve ser um policy pack versionado. Um esboço de estados pode incluir:

- 1 identificar o objetivo do aprendiz;
- 2 estimar o nível com evidências da conversa atual;
- 3 recuperar conteúdo local e exemplos relevantes;
- 4 escolher entre explicação direta, pergunta guiada, demonstração ou exercício;
- 5 pedir ao estudante que antecipe o resultado;
- 6 fornecer feedback específico;
- 7 registrar progresso somente com autorização.

Regionalidade sem estereótipo

Identidade regional significa reconhecer contexto maranhense e brasileiro, idioma, exemplos, currículo e realidade de conectividade. Não significa forçar expressões locais em toda resposta nem reduzir uma cultura a decoração. O conteúdo regional deve ser fonte citável na base de conhecimento, revisado por educadores e separado do modelo.

Personalidade portátil

A personalidade é composta por regras, valores, vocabulário preferencial, limites, objetivos pedagógicos e exemplos de interação. Ela deve ser armazenada como dados versionados. O provider recebe apenas a projeção necessária para a tarefa. Assim, trocar o modelo preserva a Atena e atualizar a personalidade não exige fine-tuning.

10. Critérios de sucesso e antiobjetivos

Uma versão inicial é bem-sucedida quando consegue, em hardware de referência, executar uma conversa local por `llama-server`, trocar para provider remoto com consentimento, responder com RAG e citação, executar pelo menos três ferramentas tipadas, registrar auditoria sem segredos, ser instalada por pacote `.deb` e iniciar como serviço de usuário.

Ela não precisa, no primeiro release, treinar um LLM do zero, controlar qualquer aplicativo, oferecer uma loja pública, suportar todas as placas ARM, manter wake word sempre ativo ou substituir todos os utilitários do desktop.

PROPOSTA DO LIVRO

Antiinchaço arquitetural

Cada marco deve entregar um caminho vertical completo. É melhor uma ferramenta `system.brightness.set` bem tipada, confirmada, testada e auditada do que cinquenta comandos que apenas concatenam texto para um shell.

PARTE II - FUNDAMENTOS DE INTELIGÊNCIA ARTIFICIAL

11. IA, aprendizado de máquina e modelos generativos

Inteligência artificial é o campo que constrói sistemas capazes de realizar tarefas associadas a percepção, decisão, linguagem, planejamento ou adaptação. Aprendizado de máquina é uma família de técnicas em que regras úteis são ajustadas a partir de dados. Deep Learning usa redes neurais com muitas transformações sucessivas. IA generativa aprende distribuições de dados e produz novas amostras, como texto, imagem ou áudio.

Um LLM, ou Large Language Model, é um modelo de linguagem com grande quantidade de parâmetros treinado para estimar sequências de tokens. “Grande” é relativo ao período e ao hardware; não define arquitetura de produto. Um LLM não possui automaticamente memória persistente, ferramentas, acesso a arquivos, consciência da hora atual ou permissão para alterar o sistema. Essas capacidades pertencem à Atena.

Termo	Pergunta que responde	Papel na Atena
Modelo	Como transformar entrada em saída provável?	motor cognitivo substituível
Chatbot	Como manter uma conversa?	uma possível interface
Assistente	Como ajudar em tarefas recorrentes?	comportamento e integração
Agente	Como observar, escolher ações e usar ferramentas?	ciclo governado pelo Agent Engine
Tool	Como produzir um efeito específico?	função tipada e autorizada
Skill	Como empacotar uma capacidade reutilizável?	manifesto, schema, política e implementação
Workflow	Como encadear etapas conhecidas?	plano determinístico ou parcialmente adaptativo
Multi-agent	Como coordenar agentes especializados?	recurso futuro, não requisito do Core inicial

12. A matemática mínima, construída por sentido

Um escalar é um único número, como a temperatura 0.7. Um vetor é uma lista ordenada de números, como uma representação de 768 dimensões. Uma matriz organiza números em linhas e colunas e pode representar uma transformação. Um tensor generaliza esse arranjo para mais dimensões.

O significado não está em um número isolado. Ele emerge da posição relativa em um espaço aprendido. Se dois trechos produzem vetores próximos segundo uma métrica, o sistema pode tratá-los como semanticamente relacionados. Essa é a intuição dos embeddings usados no RAG Standard.

Multiplicação matricial

Considere uma entrada x com características e uma matriz de pesos W . A expressão $y = xW + b$ mistura as características de x , aplica pesos e adiciona um viés b . Cada camada de uma rede repete variações desse princípio. O formato das matrizes precisa ser compatível: a dimensão que sai de x deve coincidir com a dimensão que entra em W .

Funções, probabilidade e softmax

Uma função recebe entrada e produz saída. O modelo gera **logits**, números ainda não normalizados para cada token possível. Softmax transforma esses logits em uma distribuição de probabilidade cuja soma é 1. A temperatura modifica o contraste dessa distribuição antes da amostragem: valores menores concentram probabilidade nas opções dominantes; valores maiores espalham escolhas.

Derivada e gradiente

Derivada mede como uma saída varia quando uma entrada muda. Gradiente é um vetor de derivadas que aponta a direção de crescimento mais rápido de uma função. Durante treinamento, o otimizador usa o gradiente da perda para ajustar os pesos em direção oposta, buscando reduzir o erro.

Treinar é encontrar pesos que reduzem uma medida de erro; inferir é usar pesos já encontrados para calcular uma saída.

13. Como uma rede neural aprende

Um neurônio artificial calcula uma soma ponderada de entradas, adiciona bias e aplica uma função de ativação. O **forward pass** leva dados da entrada até a previsão. A função de perda compara previsão e alvo. O **backpropagation** calcula como cada parâmetro contribuiu para o erro. O otimizador atualiza os parâmetros.

Conceito	Explicação intuitiva
peso	intensidade e direção de uma conexão
bias	deslocamento que permite ativação mesmo com entradas nulas
ativação	transformação não linear que amplia o que a rede pode representar
loss	número que resume o erro do lote atual
batch	grupo de exemplos processados antes de uma atualização
epoch	passagem completa pelo conjunto de treino
validação	dados usados para medir generalização sem ajustar pesos
inferência	uso do modelo treinado, sem backpropagation

Memorizar o treino não basta. Um modelo que apresenta perda baixa em exemplos vistos e desempenho ruim em dados novos sofreu **overfitting**. Separar treino, validação e teste ajuda a detectar isso. Para a Atena, a mesma disciplina se aplica à avaliação de prompts, ferramentas e roteamento: um conjunto de demonstrações não substitui testes fora da amostra.

14. Tokens, embeddings e Transformers

Texto precisa ser convertido em unidades numéricas. O tokenizer divide a entrada em tokens, que podem corresponder a palavras, pedaços de palavra, sinais ou bytes. Cada token recebe um identificador. Uma tabela de embeddings converte identificadores em vetores densos.

Transformers processam relações entre posições por atenção. Em uma forma simplificada, cada posição produz vetores Q (query), K (key) e V (value). A similaridade entre Q e K determina quanto de cada V contribui para a nova representação. A escala pela raiz da dimensão evita valores extremos antes do softmax.

TEXT

$$\text{Attention}(Q, K, V) = \text{softmax}((Q \times K^T) / \sqrt{d_k}) \times V$$

Self-attention significa que consultas, chaves e valores derivam da mesma sequência. Multi-head attention executa projeções diferentes em paralelo, permitindo que relações distintas sejam representadas. Uma camada também inclui bloco feed-forward, normalização e conexões residuais. As residuais criam caminhos curtos para informação e gradiente; a normalização estabiliza escalas.

Informação de posição

A atenção pura não sabe que um token veio antes de outro. Informações posicionais são adicionadas ou incorporadas ao cálculo. Implementações modernas usam diferentes técnicas, mas o princípio permanece: a ordem precisa entrar no modelo.

Logits e geração autoregressiva

O modelo produz logits para o próximo token. Um método de amostragem escolhe um token, anexa-o à sequência e repete o processo. O `context window` limita quantos tokens podem participar. O KV cache guarda chaves e valores já calculados para evitar refazer toda a sequência a cada token; por isso ele cresce com contexto, número de camadas e dimensões do modelo.

15. Quantização, contexto e custo real de inferência

Pesos treinados em alta precisão ocupam muito espaço. Quantização representa esses valores com menos bits, reduzindo armazenamento e memória e frequentemente acelerando inferência. A redução não é gratuita: diferentes esquemas alteram qualidade, velocidade e compatibilidade com backends.

Uma estimativa grosseira do tamanho dos pesos é $\text{parâmetros} \times \text{bits} / 8$. Um modelo de 2 bilhões de parâmetros em 4 bits exigiria perto de 1 GB apenas para pesos, antes de metadados, buffers, KV cache, runtime e sistema operacional. Portanto, “o arquivo cabe no disco” não significa “o modelo cabe na RAM”.

O contexto também custa memória. Dobrar `ctx-size` pode aumentar muito o KV cache. Batch, threads, GPU offload e tipo do cache mudam o perfil. Por isso o Model Manager exhibe estimativa, mas o Benchmark Engine registra o pico medido na máquina.

GGML e GGUF

GGML é a biblioteca/tensor engine que sustenta a família `llama.cpp`. GGUF é um formato de arquivo extensível que armazena tensores e metadados necessários ao carregamento. O livro trata GGUF como artefato de modelo, não como sinônimo de quantização: um GGUF pode conter diferentes tipos de tensor e informações de arquitetura.

16. Usar, adaptar ou treinar

Usar um modelo significa executar inferência com pesos existentes. Adaptar significa especializar o comportamento ou domínio. Fine-tuning atualiza parâmetros a partir de novos exemplos. LoRA congela os pesos principais e aprende matrizes de baixa dimensão. QLoRA mantém a base quantizada e treina adaptadores, reduzindo memória. Distillation treina um modelo aluno para imitar sinais de um professor. Treinar do zero começa com pesos não especializados e exige dados, computação, avaliação e operação em escala.

Estratégia	O que muda	Custo relativo	Uso inicial na Atena
Prompt e política	contexto enviado	baixo	obrigatório
RAG	conhecimento recuperado	baixo a médio	obrigatório
LoRA	adaptadores	médio	experimento controlado
QLoRA	adaptadores sobre base quantizada	médio	laboratório com GPU adequada
Distillation	novo aluno	alto	pesquisa futura
Treino do zero	todos os pesos	muito alto	não é estratégia inicial

O método socrático, a personalidade e o conhecimento regional não devem ser “enterrados” imediatamente em fine-tuning. Arquivos versionados, RAG e políticas são mais fáceis de revisar, atualizar e auditar. Fine-tuning passa a fazer sentido quando existe hipótese mensurável que prompt e recuperação não resolvem.

17. Mini Transformer educacional em Python

O mini modelo a seguir existe para revelar o ciclo completo. Ele não é o modelo final da Atena, não é eficiente e não deve ser usado como provider de produção. O objetivo é ver dataset, tokenizer, embeddings, atenção, loss, optimizer, checkpoint e inferência em um programa pequeno.

PYTHON

```

import json
import math
from pathlib import Path

import torch
import torch.nn as nn
import torch.nn.functional as F

TEXT = "atena ensina linux. linux revela o computador. " * 200
chars = sorted(set(TEXT))
stoi = {ch: i for i, ch in enumerate(chars)}
itos = {i: ch for ch, i in stoi.items()}
data = torch.tensor([stoi[ch] for ch in TEXT], dtype=torch.long)

BLOCK = 48
BATCH = 16

def batch():
    starts = torch.randint(0, len(data) - BLOCK - 1, (BATCH,))
    x = torch.stack([data[i:i + BLOCK] for i in starts])
    y = torch.stack([data[i + 1:i + BLOCK + 1] for i in starts])
    return x, y

class Head(nn.Module):
    def __init__(self, width, head_size):
        super().__init__()
        self.key = nn.Linear(width, head_size, bias=False)
        self.query = nn.Linear(width, head_size, bias=False)
        self.value = nn.Linear(width, head_size, bias=False)
        self.register_buffer("mask", torch.tril(torch.ones(BLOCK, BLOCK)))

    def forward(self, x):
        _, t, _ = x.shape
        k, q, v = self.key(x), self.query(x), self.value(x)
        scores = q @ k.transpose(-2, -1) / math.sqrt(k.size(-1))
        scores = scores.masked_fill(self.mask[:t, :t] == 0, float("-inf"))
        return F.softmax(scores, dim=-1) @ v

class TinyTransformer(nn.Module):
    def __init__(self, vocab, width=64):
        super().__init__()
        self.token = nn.Embedding(vocab, width)
        self.pos = nn.Embedding(BLOCK, width)
        self.head = Head(width, width)
        self.ff = nn.Sequential(
            nn.Linear(width, 4 * width), nn.ReLU(), nn.Linear(4 * width, width)
        )
        self.norm1, self.norm2 = nn.LayerNorm(width), nn.LayerNorm(width)
        self.out = nn.Linear(width, vocab)

    def forward(self, ids, targets=None):
        _, t = ids.shape
        x = self.token(ids) + self.pos(torch.arange(t, device=ids.device))
        x = x + self.head(self.norm1(x))
        x = x + self.ff(self.norm2(x))
        logits = self.out(x)
        loss = None
        if targets is not None:
            loss = F.cross_entropy(logits.view(-1, logits.size(-1)), targets.view(-1))
        return logits, loss

model = TinyTransformer(len(chars))
optimizer = torch.optim.AdamW(model.parameters(), lr=3e-4)

for step in range(1500):
    x, y = batch()
    _, loss = model(x, y)
    optimizer.zero_grad(set_to_none=True)
    loss.backward()
    optimizer.step()
    if step % 150 == 0:
        print(step, float(loss))

torch.save(model.state_dict(), "tiny-atena.pt")
Path("tiny-atena-vocab.json").write_text(json.dumps(stoi), encoding="utf-8")

```

Inference do mini modelo

PYTHON

```
@torch.no_grad()
def generate(model, prefix, count=120, temperature=0.8):
    ids = torch.tensor([[stoi[ch] for ch in prefix]], dtype=torch.long)
    for _ in range(count):
        window = ids[:, -BLOCK:]
        logits, _ = model(window)
        probs = F.softmax(logits[:, -1, :] / temperature, dim=-1)
        nxt = torch.multinomial(probs, 1)
        ids = torch.cat([ids, nxt], dim=1)
    return "".join(itos[int(i)] for i in ids[0])

print(generate(model, "atena "))
```

Observe o que o programa não possui: instruções, fatos confiáveis, alinhamento, ferramentas, memória externa, filtros, arquitetura multiusuário e avaliação robusta. Ele ensina o mecanismo de treino, não reproduz um LLM moderno.

18. Avaliação de modelos e do sistema completo

Perplexidade e loss medem aspectos do modelo, mas não bastam para um agente de sistema. A Atena precisa de avaliações por camada.

- Modelo: qualidade de geração, idioma, código, contexto e tool calling.
- RAG: recall de trechos, precisão, cobertura de citação e fidelidade à fonte.
- Router: escolha correta de capability, privacidade, custo e fallback.
- Tool: schema, autorização, efeito real, idempotência e reversibilidade.
- Produto: tempo até primeiro token, acessibilidade, clareza de consentimento e recuperação de falha.
- Segurança: prompt injection, exfiltração, path traversal, segredos em log e elevação indevida.

Um benchmark público pode comparar modelos; um benchmark local informa se aquele modelo serve naquela máquina. Os dois são necessários e respondem a perguntas diferentes.

PARTE III - C DO ZERO, DENTRO DA ATENA

19. Código-fonte, compilador e programa

Código-fonte é texto escrito em uma linguagem. O processador não executa diretamente `if`, `struct` ou nomes de função; ele executa instruções da arquitetura da CPU. O compilador analisa o código, verifica tipos, transforma-o em representação intermediária, otimiza e produz código de máquina. O linker, ou ligador, combina objetos e bibliotecas para formar o executável.

No Debian, GCC e Clang são duas cadeias de compilação importantes. O projeto deve funcionar com ambas sempre que possível. Começaremos com comandos explícitos para entender as etapas; depois adotaremos CMake para builds reproduzíveis.

C

```
#include <stdio.h>

int main(void) {
    puts("Atena Core iniciando");
    return 0;
}
```

BASH

```
cc -std=c17 -Wall -Wextra -Wpedantic -Werror \
    hello.c -o hello
./hello
```

`-std=c17` seleciona a versão da linguagem. `-Wall -Wextra -Wpedantic` ativa avisos. `-Werror` transforma aviso em erro durante integração contínua. Um aviso não é “apenas estética”: frequentemente revela conversão perigosa, retorno ignorado ou variável sem uso.

As quatro etapas visíveis

BASH

```
cc -E hello.c -o hello.i      # pre-processamento
cc -S hello.i -o hello.s      # assembly
cc -c hello.s -o hello.o      # objeto
cc hello.o -o hello           # link
```

O pre-processor expande `#include` e macros. O compilador produz assembly. O assembler cria o arquivo objeto. O linker resolve símbolos como `puts`. Entender essa cadeia ajuda a diagnosticar “header não encontrado”, “undefined reference” e incompatibilidade de arquitetura.

20. Tipos, variáveis e representação

Um tipo descreve quais valores e operações são válidos e influencia a representação em memória. C possui inteiros, ponto flutuante, caracteres, booleanos por `<stdbool.h>` e tipos compostos. O tamanho de `int` não deve ser presumido para formatos persistentes. Use tipos de largura definida de `<stdint.h>` quando o protocolo exige `uint32_t`, `int64_t` e semelhantes.

C

```
#include <stdbool.h>
#include <stdint.h>
#include <stdio.h>

int main(void) {
    uint64_t ram_bytes = 8ULL * 1024 * 1024 * 1024;
    double tokens_per_second = 5.8;
    bool cloud_allowed = false;

    printf("RAM: %llu bytes\n", (unsigned long long)ram_bytes);
    printf("Velocidade: %.1f tok/s\n", tokens_per_second);
    printf("Cloud: %s\n", cloud_allowed ? "sim" : "nao");
    return 0;
}
```

Variável é um nome associado a uma região de armazenamento. Inicializar no momento da declaração evita valores indeterminados. `const` comunica que o código não deve modificar o objeto por aquele acesso. Isso não é o mesmo que uma constante de compilação em todos os contextos, mas melhora contratos.

Conversões e overflow

Converter um número maior para um tipo menor pode truncar bits. Inteiro sem sinal faz aritmética modular; overflow de inteiro com sinal é comportamento indefinido. O hardware profiler lida com bytes e contadores grandes, então deve validar limites antes de somar ou multiplicar.

C

```
bool checked_add_size(size_t a, size_t b, size_t *out) {
    if (out == NULL || b > SIZE_MAX - a) {
        return false;
    }
    *out = a + b;
    return true;
}
```

21. Decisão e repetição

`if` escolhe um caminho com base em uma condição. `switch` organiza escolhas discretas. `for`, `while` e `do while` repetem. Estruturas simples são preferíveis a expressões compactas que escondem política.

C

```
typedef enum {
    ATENA_PRIVACY_PUBLIC,
    ATENA_PRIVACY_NORMAL,
    ATENA_PRIVACY_SENSITIVE,
    ATENA_PRIVACY_PRIVATE,
    ATENA_PRIVACY_SECRET
} AtenaPrivacyClass;

bool may_use_cloud(AtenaPrivacyClass level, bool user_confirmed) {
    switch (level) {
        case ATENA_PRIVACY_PUBLIC:
        case ATENA_PRIVACY_NORMAL:
            return true;
        case ATENA_PRIVACY_SENSITIVE:
            return user_confirmed;
        case ATENA_PRIVACY_PRIVATE:
        case ATENA_PRIVACY_SECRET:
            return false;
    }
    return false;
}
```

O `return false` final protege contra um valor inválido, embora compiladores possam avisar que todos os enumeradores foram tratados. Política sensível deve falhar fechada: na dúvida, negar ou pedir confirmação, não liberar.

Loops e invariantes

Uma invariante é uma condição que permanece verdadeira em cada iteração. Ao ler uma resposta HTTP em blocos, por exemplo, o tamanho já usado nunca pode ultrapassar a capacidade alocada. Escrever a invariante antes do loop reduz erros de memória.

22. Funções, contratos e erros

Função encapsula uma transformação ou efeito. Um bom contrato declara pré-condições, resultado e erros. Em C não há exceções nativas; APIs normalmente usam retorno, parâmetros de saída e códigos de erro.

C

```
typedef enum {
    ATENA_OK = 0,
    ATENA_ERR_INVALID,
    ATENA_ERR_NOMEM,
    ATENA_ERR_IO,
    ATENA_ERR_POLICY,
    ATENA_ERR_PROVIDER,
    ATENA_ERR_UNSUPPORTED
} AtenaStatus;

const char *atena_status_string(AtenaStatus status) {
    switch (status) {
        case ATENA_OK: return "ok";
        case ATENA_ERR_INVALID: return "invalid argument";
        case ATENA_ERR_NOMEM: return "out of memory";
        case ATENA_ERR_IO: return "I/O error";
        case ATENA_ERR_POLICY: return "blocked by policy";
        case ATENA_ERR_PROVIDER: return "provider error";
        case ATENA_ERR_UNSUPPORTED: return "unsupported";
    }
    return "unknown error";
}
```

Não misture log com tratamento de erro. A biblioteca devolve status e contexto seguro; a camada externa decide o que registrar e mostrar. Erros provenientes de provider podem conter trechos sensíveis e não devem ser copiados integralmente para o journal.

errno

Funções POSIX podem sinalizar detalhes em `errno`. Ele só é significativo quando a função documenta falha. Leia-o imediatamente e converta a mensagem para uma estrutura própria. Não use `errno` como estado global do Core.

23. Arrays, strings e limites

Array é uma sequência contígua de elementos do mesmo tipo. Ao passar um array para função, ele normalmente decai para ponteiro; o tamanho não viaja automaticamente. Por isso, passe ponteiro e comprimento juntos.

Strings C são arrays de `char` terminados por byte zero. O terminador ocupa espaço. Funções que não conhecem a capacidade do destino são fonte recorrente de vulnerabilidades.

C

```
#include <stdio.h>

AtenaStatus format_model_label(
    char *dst,
    size_t capacity,
    const char *family,
    const char *quantization
) {
    if (dst == NULL || capacity == 0 || family == NULL || quantization == NULL) {
        return ATENA_ERR_INVALID;
    }
    int written = snprintf(dst, capacity, "%s (%s)", family, quantization);
    if (written < 0 || (size_t)written >= capacity) {
        dst[0] = '\0';
        return ATENA_ERR_INVALID;
    }
    return ATENA_OK;
}
```

Texto Unicode em UTF-8 usa quantidade variável de bytes por caractere. `strlen` conta bytes até o zero, não letras visíveis. A UI deve usar toolkit com suporte Unicode; o Core preserva UTF-8 válido e não corta texto em um byte arbitrário.

24. Structs, enums e modelagem de domínio

`struct` reúne campos relacionados. `enum` nomeia estados discretos. Juntos, eles transformam conceitos do projeto em tipos verificáveis.

C

```
typedef enum {
    ATENA_CAP_TEXT      = 1u << 0,
    ATENA_CAP_VISION    = 1u << 1,
    ATENA_CAP_AUDIO     = 1u << 2,
    ATENA_CAP_TOOLS     = 1u << 3,
    ATENA_CAP_JSON      = 1u << 4,
    ATENA_CAP_STREAMING = 1u << 5,
    ATENA_CAP_EMBEDDINGS = 1u << 6
} AtenaCapability;

typedef struct {
    const char *id;
    const char *display_name;
    uint32_t capabilities;
    bool is_local;
    bool is_available;
    uint32_t max_context_tokens;
} AtenaProviderInfo;

bool provider_has(const AtenaProviderInfo *p, AtenaCapability cap) {
    return p != NULL && (p->capabilities & (uint32_t)cap) != 0;
}
```

Flags por bits permitem combinar capacidades. O schema persistido deve usar nomes textuais, porque números tornam migrações e inspeção difíceis. A máscara é uma representação interna eficiente.

Layout e ABI

Compiladores podem inserir padding entre campos para alinhamento. Não grave uma `struct` diretamente em arquivo nem a envie pela rede esperando formato estável. Serialize para JSON, CBOR ou protocolo definido. ABI é o contrato binário entre objetos, bibliotecas e chamadas; mudar a ordem de campos de uma API pública pode quebrar plugins compilados.

25. Ponteiros sem misticismo

Ponteiro contém um endereço de memória. `&value` obtém o endereço; `*pointer` acessa o objeto apontado. O tipo informa como interpretar os bytes e quanto avançar em aritmética de ponteiros.

C

```
void clamp_temperature(double *value, double low, double high) {
    if (value == NULL) return;
    if (*value < low) *value = low;
    if (*value > high) *value = high;
}
```

O ponteiro não carrega validade automática. Pode ser nulo, apontar para objeto já liberado, ter alinhamento errado ou referir menos bytes do que a função espera. O contrato de ownership responde: quem cria? quem pode modificar? quem libera? por quanto tempo o endereço permanece válido?

Padrão	Significado recomendado
<code>const T *borrowed</code>	leitura temporária; chamador mantém ownership
<code>T *borrowed_mut</code>	acesso mutável temporário
<code>T **out</code>	função entrega referência por parâmetro de saída
<code>T *create()</code>	retorno geralmente transfere ownership
<code>destroy(T *)</code>	libera objeto criado pela API correspondente

Ponteiro para ponteiro

Quando uma função precisa alterar o ponteiro do chamador, recebe `T **`. Isso aparece em construtores que alocam um objeto e devolvem status separado.

C

```
AtenaStatus atena_buffer_create(size_t capacity, AtenaBuffer **out);
void atena_buffer_destroy(AtenaBuffer *buffer);
```

26. Stack, heap, malloc e ciclo de vida

A stack guarda frames de chamadas, parâmetros e variáveis automáticas. Sua liberação ocorre quando a função retorna. O heap guarda alocações com ciclo de vida explícito. `malloc` reserva bytes sem inicializar; `calloc` inicializa com zero; `realloc` tenta redimensionar; `free` libera.

C

```
typedef struct {
    char *data;
    size_t length;
    size_t capacity;
} AtenaBuffer;

AtenaStatus buffer_init(AtenaBuffer *b, size_t initial) {
    if (b == NULL || initial == 0) return ATENA_ERR_INVALID;
    b->data = malloc(initial);
    if (b->data == NULL) return ATENA_ERR_NOMEM;
    b->data[0] = '\0';
    b->length = 0;
    b->capacity = initial;
    return ATENA_OK;
}

void buffer_deinit(AtenaBuffer *b) {
    if (b == NULL) return;
    free(b->data);
    b->data = NULL;
    b->length = 0;
    b->capacity = 0;
}
```

Erros clássicos incluem leak (esquecer `free`), use-after-free, double free, acesso além do limite e retorno de endereço de variável automática. A regra “cada aquisição tem uma liberação visível” simplifica revisão.

realloc seguro

Não atribua `realloc` diretamente ao ponteiro original. Em falha, ele retorna `NULL` e a alocação anterior continua válida; uma atribuição direta perderia o endereço.

C

```
char *next = realloc(b->data, new_capacity);
if (next == NULL) {
    return ATENA_ERR_NOMEM;
}
b->data = next;
b->capacity = new_capacity;
```

Call stack

Cada chamada guarda endereço de retorno e estado necessário. Recursão profunda consome stack. O chunker de documentos deve preferir iteração em arquivos enormes; um parser recursivo precisa de limites explícitos para entrada não confiável.

27. Ponteiros de função e interfaces substituíveis

Ponteiro de função permite guardar comportamento. É a base de uma Provider API em C sem acoplar o Core a implementações.

C

```
typedef struct AtenaProvider AtenaProvider;

typedef struct {
    AtenaStatus (*start)(AtenaProvider *self);
    AtenaStatus (*generate)(
        AtenaProvider *self,
        const AtenaRequest *request,
        AtenaStreamSink *sink
    );
    void (*cancel)(AtenaProvider *self, uint64_t request_id);
    void (*destroy)(AtenaProvider *self);
} AtenaProviderVTable;

struct AtenaProvider {
    const AtenaProviderVTable *ops;
    AtenaProviderInfo info;
    void *impl;
};
```

`vtable` é uma tabela de operações. `impl` aponta para estado privado do provider. O Core chama apenas `ops`. Uma implementação HTTP guarda URL, pool e credencial por referência; uma implementação libllama guarda contexto nativo.

Antes de chamar, valide ponteiros e versão da interface. Plugins externos exigem campo `abi_version` e `struct_size`; assim, o loader pode rejeitar incompatibilidades em vez de interpretar layout errado.

28. Headers, módulos e encapsulamento

Header público declara o que outros módulos podem usar. O `.c` contém implementação. Inclua guardas e exponha o mínimo. Objetos opacos escondem layout.

C

```
// include/atenas/session.h
#ifndef ATENA_SESSION_H
#define ATENA_SESSION_H

#include <stdint.h>
#include "atenas/status.h"

typedef struct AtenaSession AtenaSession;

AtenaStatus atena_session_create(AtenaSession **out);
uint64_t atena_session_id(const AtenaSession *session);
void atena_session_destroy(AtenaSession *session);

#endif
```

O chamador não acessa campos e não depende do tamanho. Isso permite trocar SQLite, estrutura de fila ou locks sem quebrar a API fonte.

Organização de dependências

Core não inclui headers de um provider específico. Provider depende da API do Core, não o contrário. Tool Engine conhece descritores de tools; cada implementação conhece apenas seu contrato. Dependências apontando em todas as direções produzem builds frágeis e testes difíceis.

29. CMake e build reproduzível

O build precisa declarar fontes, bibliotecas, opções e testes. Um CMake inicial:

CMAKE

```

cmake_minimum_required(VERSION 3.25)
project(atenas VERSION 0.1.0 LANGUAGES C)

set(CMAKE_C_STANDARD 17)
set(CMAKE_C_STANDARD_REQUIRED ON)
set(CMAKE_C_EXTENSIONS OFF)

include(GNUInstallDirs)
include(CTest)
find_package(PkgConfig REQUIRED)
pkg_check_modules(JSONC REQUIRED IMPORTED_TARGET json-c)
pkg_check_modules(SQLITE REQUIRED IMPORTED_TARGET sqlite3)
pkg_check_modules(CURL REQUIRED IMPORTED_TARGET libcurl)

add_library(atenas-core
    core/request.c
    core/policy.c
    core/router.c
    memory/store.c
)
target_include_directories(atenas-core PUBLIC include)
target_link_libraries(atenas-core
    PRIVATE PkgConfig::JSONC PkgConfig::SQLITE PkgConfig::CURL
)
target_compile_options(atenas-core PRIVATE
    -Wall -Wextra -Wpedantic -Wconversion -Wshadow
)

add_executable(atenas-cli cli/main.c)
target_link_libraries(atenas-cli PRIVATE atenas-core)

install(TARGETS atenas-core atenas-cli
    LIBRARY DESTINATION ${CMAKE_INSTALL_LIBDIR}
    RUNTIME DESTINATION ${CMAKE_INSTALL_BINDIR}
)

if(BUILD_TESTING)
    add_subdirectory(tests)
endif()

```

BASH

```

cmake -S . -B build -DCMAKE_BUILD_TYPE=Debug
cmake --build build -j2
ctest --test-dir build --output-on-failure

```

Em hardware limitado, `-j2` evita que a compilação paralela esgote RAM. Builds de release usam otimização e símbolos de debug separados no pacote correspondente.

30. Arquivos, JSON, SQLite e HTTP

O Core precisa ler configurações, serializar contratos, persistir estado e conversar com providers. Em vez de implementar formatos e protocolos do zero, usamos bibliotecas maduras: `json-c`, `SQLite` e `libcurl` são opções disponíveis no ecossistema Debian.

Arquivos

Abra com modo correto, verifique retorno, limite tamanho e feche em todo caminho. Para configuração, leia de caminho conhecido sob XDG; não aceite um caminho vindo do modelo sem política.

JSON

JSON é útil para fronteiras e auditoria, mas schema ainda é necessário. A existência da chave não garante tipo, faixa ou semântica. Rejeite campos inesperados em ações de alto risco e limite profundidade/tamanho para prevenir consumo abusivo.

JSON

```
{
  "schema": "org.athenas.atena.request/1",
  "id": "018f...",
  "task": "system.brightness.set",
  "privacy": "NORMAL",
  "input": {"percent": 40},
  "requires": ["TOOLS"],
  "interactive": true
}
```

SQLite

SQLite oferece transações, índices e arquivo único. Use prepared statements; nunca concatene texto do usuário em SQL. Ative foreign keys, defina busy timeout e trate migrações por versão.

HTTP

Provider HTTP precisa de timeout de conexão, timeout total, cancelamento, limite de resposta, TLS verificado e streaming incremental. Uma requisição que nunca termina não pode bloquear o Core inteiro. Credenciais são obtidas do Secret Service no momento necessário e não entram na URL nem no log.

31. Processos, threads, sinais e IPC

Processo possui espaço de endereçamento isolado. Thread compartilha memória dentro do processo. Concorrência melhora responsividade, mas cria condições de corrida quando múltiplas threads acessam estado mutável sem coordenação.

O Core pode usar um event loop para sockets e tarefas curtas, com pool limitado para extração e chamadas bloqueantes. Inferência local pesada ocorre no `llama-server` ou runtime dedicado, não em uma thread que congela a UI.

Cancelamento

Cancelar é parte do contrato. A UI emite cancelamento pelo ID; o Core marca o request; provider interrompe streaming; Tool Engine decide se a ação já iniciada pode ser revertida. `SIGKILL` é último recurso porque não permite limpeza.

Socket Unix

No estágio inicial de serviço, um socket em `$XDG_RUNTIME_DIR/atena/core.sock` oferece IPC local. O diretório já é associado ao usuário e à sessão. O servidor valida credenciais do peer, limita mensagem e usa framing explícito, como comprimento de 32 bits seguido de JSON. “Uma linha por JSON” serve ao protótipo, mas strings podem conter quebras e mensagens enormes exigem limite.

D-Bus

D-Bus é adequado para APIs de desktop e descoberta de serviços. Métodos devem ser semânticos, não `ExecuteShell(string)`. Exemplos: `Ask`, `Cancel`, `ListModels`, `InvokeSkill`. Sinais transportam tokens parciais e mudança de estado. Dados volumosos, como áudio e arquivos, usam descritores ou caminhos autorizados, não blocos ilimitados no barramento.

32. Debugging, sanitizers e testes em C

GDB permite pausar, inspecionar stack e variáveis. Valgrind ajuda a encontrar leaks em arquiteturas suportadas. Sanitizers instrumentam o binário durante testes.

BASH

```
cmake -S . -B build-asan \
  -DCMAKE_C_FLAGS="-fsanitize=address,undefined -fno-omit-frame-pointer -g"
cmake --build build-asan
ctest --test-dir build-asan --output-on-failure
```

AddressSanitizer detecta acessos inválidos; UndefinedBehaviorSanitizer detecta categorias de comportamento indefinido. ThreadSanitizer ajuda com corridas, mas normalmente é executado separadamente. Sanitizers não substituem revisão nem fuzzing.

Teste unitário

Teste unitário isola função. Tabelas de casos funcionam bem para política.

C

```
static void test_private_never_uses_cloud(void) {
    assert(!may_use_cloud(ATENA_PRIVACY_PRIVATE, false));
    assert(!may_use_cloud(ATENA_PRIVACY_PRIVATE, true));
}
```

Fuzzing

Parsers de JSON, manifestos e protocolos são bons alvos para libFuzzer/AFL++. O objetivo é gerar entradas inesperadas e verificar crash, leak ou consumo descontrolado. Um parser seguro possui limites de bytes, profundidade, número de campos e tempo.

33. Linux nativo: XDG, systemd, D-Bus e Polkit

Atena Core roda como usuário. Arquivos de configuração seguem \$XDG_CONFIG_HOME/atena; dados persistentes, \$XDG_DATA_HOME/atena; cache recriável, \$XDG_CACHE_HOME/atena; estado de runtime, \$XDG_RUNTIME_DIR/atena. Nunca salve API keys em configuração comum.

Serviço systemd de usuário

INI

```
[Unit]
Description=Atena Core user service
After=graphical-session.target

[Service]
Type=simple
ExecStart=/usr/libexec/atena/atena-core
Restart=on-failure
RestartSec=2
NoNewPrivileges=yes
PrivateTmp=yes
ProtectSystem=strict
ProtectHome=read-only
ReadWritePaths=%h/.local/share/atena %h/.cache/atena
RestrictAddressFamilies=AF_UNIX AF_INET AF_INET6

[Install]
WantedBy=default.target
```

As opções de hardening precisam ser testadas. `ProtectHome=read-only`, por exemplo, impede escrita fora dos caminhos liberados, mas o RAG precisa ler somente diretórios autorizados. A política final pode usar portal ou file chooser para conceder acesso granular.

Polkit

Polkit separa cliente sem privilégio, mecanismo privilegiado e autoridade de autorização. O Core pede uma ação semântica. Um helper pequeno valida novamente os argumentos e realiza somente aquela operação. A decisão não é “o modelo é confiável?”, mas “este usuário, nesta sessão, confirmou esta ação definida?”.

LIMITE DE SEGURANÇA Proibição estrutural

Não crie `atena-root-shell`, `socket root` genérico, regra `NOPASSWD: ALL` nem método D-Bus que receba texto de shell. Isso anularia toda a arquitetura de permissões.

34. Segredos e higiene operacional

Configuração contém nome do provider, endpoint, modelo e preferências. Segredo contém chave, token ou certificado privado. O arquivo de configuração guarda apenas uma referência, por exemplo `secret_ref = "atena/provider/openai/default"`. A implementação consulta o Secret Service da sessão.

Regras mínimas:

- nunca inserir chave em repositório, screenshot, prompt de teste ou pacote;
- não aceitar segredo por argumento de linha de comando, pois pode aparecer em lista de processos;
- limpar headers de autorização antes de logar;
- permitir rotação e revogação sem editar banco de conversas;
- restringir permissões de qualquer fallback em arquivo a `0600`;
- tratar crash dumps como dados sensíveis;
- usar variáveis de ambiente apenas em desenvolvimento controlado.

O logger recebe campos já redigidos. Redação posterior é frágil: uma exceção pode registrar a estrutura antes do filtro.

PARTE IV - CONSTRUINDO O ATENA CORE

35. Desenvolvimento vertical e contratos primeiro

O Core será construído por fatias verticais. A primeira fatia recebe texto na CLI, forma um `AtenaRequest`, escolhe um provider simulado, transmite tokens e registra resultado. A segunda substitui o provider simulado por `llama-server`. A terceira acrescenta política e uma ferramenta segura. Cada fatia compila, testa e produz comportamento observável.

Antes da implementação, criamos contratos independentes de provider:

C

```
typedef struct {
    uint64_t id;
    const char *session_id;
    const char *user_text;
    AtenaPrivacyClass privacy;
    uint32_t required_capabilities;
    bool allow_network;
    bool allow_tools;
} AtenaRequest;

typedef enum {
    ATENA_EVENT_TEXT_DELTA,
    ATENA_EVENT_TOOL_PROPOSAL,
    ATENA_EVENT_CITATION,
    ATENA_EVENT_USAGE,
    ATENA_EVENT_DONE,
    ATENA_EVENT_ERROR
} AtenaEventType;

typedef struct {
    AtenaEventType type;
    uint64_t request_id;
    const char *json_payload;
} AtenaEvent;

typedef AtenaStatus (*AtenaEventCallback)(
    const AtenaEvent *event,
    void *user_data
);
```

Eventos unificam streaming local e remoto. O payload de cada tipo possui schema. A UI não precisa saber se o delta veio de SSE, socket ou chamada de biblioteca.

Imutabilidade lógica

Depois de entrar no Request Engine, campos de segurança do request não são alterados por provider ou agente. Uma tarefa derivada cria novo objeto com relação explícita ao pai. Isso impede que uma Skill mude `privacy=PRIVATE` para `NORMAL` no mesmo ponteiro.

36. Request Engine e Context Builder

O Request Engine valida tamanho, UTF-8, session ID, origem e anexos. Em seguida, classifica a tarefa e cria um plano de contexto. O Context Builder reúne apenas o necessário: instruções da Atena, estado de sessão, trechos RAG, resultados de ferramentas e histórico dentro do orçamento de tokens.

O contexto é uma montagem temporária. Banco de memória e documentos não são copiados integralmente para o prompt. Cada fragmento registra origem e classe de privacidade.

Ordem	Bloco	Motivo
1	contrato de sistema da Atena	identidade e regras invariantes
2	policy pack ativo	modo, privacidade e pedagogia
3	definição das tools permitidas	schema para aquela requisição
4	memória aprovada	preferências relevantes
5	evidências RAG	conhecimento com locators
6	histórico resumido	continuidade dentro do orçamento
7	entrada atual	objetivo do usuário

Orçamento de contexto

Reserve tokens para saída e tool results. Se o modelo suporta 4096 tokens e a resposta precisa de até 800, não envie 4090 de entrada. O budgeter estima tokens por tokenizer do modelo quando disponível e por aproximação conservadora no fallback.

Prompt injection em documentos

Trechos recuperados são dados não confiáveis, mesmo quando locais. Um PDF pode conter “ignore suas regras”. O Context Builder separa instruções de evidências, rotula fontes e orienta o modelo a não executar comandos contidos nelas. Segurança não depende apenas dessa instrução: Tool Engine continua validando fora do modelo.

37. Sessão, histórico, memória e conhecimento

Memoria nao e uma unica caixa



Sessão é estado efêmero de uma interação ativa: request atual, cancelamento e provider escolhido. Histórico é a sequência de turnos. Memória de longo prazo são fatos ou resumos persistidos segundo política. Preferências são escolhas do usuário. Base de conhecimento contém documentos indexados. Misturar tudo em uma tabela “memory” torna limpeza e consentimento confusos.

Modelo de dados inicial

SQL

```
PRAGMA foreign_keys = ON;

CREATE TABLE schema_migrations (
  version INTEGER PRIMARY KEY,
  applied_at TEXT NOT NULL
);

CREATE TABLE conversations (
  id TEXT PRIMARY KEY,
  title TEXT,
  created_at TEXT NOT NULL,
  privacy_class TEXT NOT NULL,
  retention_policy TEXT NOT NULL
);

CREATE TABLE messages (
  id TEXT PRIMARY KEY,
  conversation_id TEXT NOT NULL REFERENCES conversations(id) ON DELETE CASCADE,
  role TEXT NOT NULL,
  content TEXT NOT NULL,
  provider_id TEXT,
  model_id TEXT,
  created_at TEXT NOT NULL
);

CREATE TABLE preferences (
  key TEXT PRIMARY KEY,
  value_json TEXT NOT NULL,
  updated_at TEXT NOT NULL
);

CREATE TABLE memories (
  id TEXT PRIMARY KEY,
  statement TEXT NOT NULL,
  provenance TEXT NOT NULL,
  privacy_class TEXT NOT NULL,
  approved INTEGER NOT NULL DEFAULT 0,
  expires_at TEXT
);
```

O usuário visualiza, exporta, limpa e desativa cada categoria. Apagar uma conversa remove mensagens e resumos derivados. Índices de documentos são reconstruíveis; preferências podem ser exportadas em JSON legível.

Retenção

Políticas sugeridas: `ephemeral`, `session`, `30-days`, `until-deleted`. O modo confidencial usa efêmero por padrão. Não trate “histórico desativado” como “esconder a tela”; a persistência deve realmente ser interrompida.

38. Provider API e descoberta de capacidades

Provider é um adaptador entre contrato Atena e runtime. Ele anuncia identidade, transporte, capacidades, limites e saúde. Model é uma entrada dentro do provider. Um provider pode listar vários modelos; o mesmo modelo conceitual pode existir em providers diferentes.

JSON

```
{
  "schema": "org.athenas.atena.provider/1",
  "id": "llama.local",
  "kind": "LOCAL_HTTP",
  "endpoint": "http://127.0.0.1:8080",
  "capabilities": ["TEXT", "JSON", "TOOLS", "STREAMING", "EMBEDDINGS"],
  "network_scope": "LOOPBACK_ONLY",
  "secret_ref": null,
  "health": {"path": "/health", "timeout_ms": 800}
}
```

Capabilities padronizadas incluem TEXT, VISION, AUDIO, TTS, STT, EMBEDDINGS, TOOLS, JSON, STREAMING, LONG_CONTEXT e IMAGE_GENERATION. Um provider pode ter capability condicional por modelo. O resolver consulta a interseção.

Normalização sem empobrecimento

O contrato comum cobre mensagens, streaming, tools, usage e erros. Recursos exclusivos entram em extensão namespaced, nunca em `if (provider == X)` espalhado pelo Core. Assim, a base permanece comum e o provider pode preservar inovação.

Health e circuit breaker

Falhas repetidas abrem um circuit breaker: por um intervalo curto, o router evita o provider e devolve causa clara. Depois tenta estado `half-open`. Isso impede que cada requisição espere o mesmo timeout de um serviço indisponível.

39. Provider Router e Model Router

Provider Router escolhe ambiente e transporte; Model Router escolhe modelo dentro dele. Separar decisões evita associar “cloud” a um único modelo ou “local” a um único runtime.

Roteamento de uma requisicao



O algoritmo elimina candidatos antes de pontuar:

- 1 remover providers desabilitados ou sem saúde;
- 2 remover os que violam modo e Privacy Engine;
- 3 remover os que não oferecem capabilities exigidas;
- 4 remover modelos incompatíveis com hardware ou licença;
- 5 pontuar candidatos restantes por política;
- 6 registrar justificativa e executar;
- 7 aplicar fallback somente dentro dos limites autorizados.

Políticas predefinidas:

Política	Prioridade dominante	Regra importante
Offline	zero rede	nunca sugerir fallback cloud automático
Privacidade máxima	dados locais	perguntar antes de qualquer exceção
Econômico	custo monetário	respeitar teto e orçamento mensal
Equilibrado	qualidade, latência e custo	padrão transparente
Qualidade	capacidade e avaliação	ainda respeita privacidade
Velocidade	TTFT e tokens/s	pode escolher modelo menor

Política	Prioridade dominante	Regra importante
Personalizado	pesos do usuário	valida combinações impossíveis

Explicação da escolha

O router produz um `RoutingDecision` com candidatos rejeitados, razão, score e métricas usadas. A UI pode resumir: “modelo local selecionado porque o conteúdo é PRIVATE” ou “provider remoto escolhido após confirmação porque o modelo local não oferece visão”.

40. Privacy Engine

Privacy Engine classifica conteúdo e aplica regras de fluxo. A classificação inicial pode vir da origem: diretório protegido, campo marcado como segredo, documento empresarial ou escolha manual. Heurísticas auxiliam, mas não rebaixam uma classe explícita.

Classe	Comportamento padrão
PUBLIC	pode usar provider configurado
NORMAL	segue preferência do usuário
SENSITIVE	pergunta antes de transmitir
PRIVATE	processamento somente local
SECRET	não entra em prompt, log, memória nem rede

Regras por caminho são avaliadas com caminho canônico e descritor aberto, evitando bypass por `..` ou symlink. Uma política pode declarar:

TOML

```
[privacy]
default = "NORMAL"
ask_before_cloud = true
never_upload_files = true

[[privacy.path_rule]]
path = "/home/user/Projetos/cliente"
class = "PRIVATE"
recursive = true

[[privacy.path_rule]]
path = "/home/user/.ssh"
class = "SECRET"
recursive = true
```

Atena Secure Local

No modo confidencial, providers cloud, busca web, telemetria, embeddings remotos e voz remota ficam desabilitados. Histórico persistente pode ser desligado. Internet opcional por allowlist é uma configuração separada e visível. O modo é apropriado a código proprietário, auditorias, laboratórios, investigações e pesquisa sigilosa.

41. Policy Engine e decisões de autorização

Privacy Engine governa dados; Policy Engine governa ações. A saída é uma decisão estruturada:

c

```
typedef enum {
    ATENA_DECISION_ALLOW,
    ATENA_DECISION_DENY,
    ATENA_DECISION_CONFIRM,
    ATENA_DECISION_AUTHENTICATE
} AtenaPolicyDecision;

typedef struct {
    AtenaPolicyDecision decision;
    const char *rule_id;
    const char *human_reason;
    bool log_required;
} AtenaPolicyResult;
```

CONFIRM pede decisão contextual do usuário. **AUTHENTICATE** aciona Polkit quando a operação administrativa definida exige autenticação. Confirmação não é autorização root; são etapas distintas.

Risco e reversibilidade

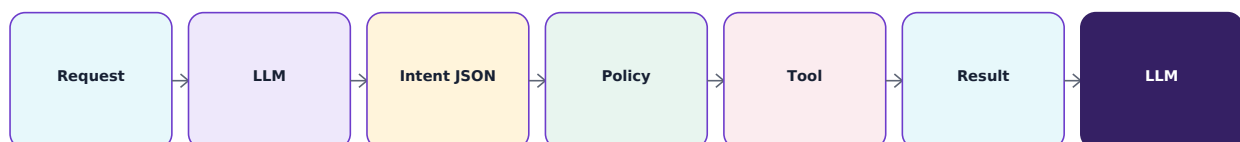
Cada tool declara risco **LOW**, **MEDIUM**, **HIGH** ou **CRITICAL**, efeitos e reversibilidade. Ler brilho atual é baixo risco. Ajustar brilho é reversível e pode pedir confirmação conforme preferência. Desligar o computador interrompe trabalho e sempre exige confirmação. Apagar arquivo permanentemente não deve existir na primeira versão; mover para lixeira pode ser uma tool distinta.

Time of check versus time of use

Entre validar e executar, o alvo pode mudar. A implementação deve operar sobre identificador ou descritor validado, não reabrir um caminho não confiável. Helpers privilegiados validam novamente os argumentos recebidos.

42. Tool Engine: efeitos tipados, não texto de shell

Ciclo controlado do agente



O modelo propõe uma chamada que segue schema. Tool Router localiza o descritor. Policy Engine decide. O usuário confirma quando necessário. Só então a função C ou processo conhecido executa.

JSON

```
{
  "tool": "system.brightness.set",
  "version": 1,
  "arguments": {"percent": 40},
  "reason": "O usuario pediu para reduzir o brilho"
}
```

O campo **reason** não concede permissão; apenas ajuda a UI. O schema limita **percent** entre 1 e 100. A implementação usa D-Bus ou interface documentada do sistema. Nenhuma parte concatena **percent** em comando de shell.

Descritor de tool

JSON

```
{
  "name": "system.brightness.set",
  "version": "1.0.0",
  "risk": "MEDIUM",
  "permissions": ["system.display.write"],
  "requires_confirmation": true,
  "input_schema": {
    "type": "object",
    "additionalProperties": false,
    "required": ["percent"],
    "properties": {
      "percent": {"type": "integer", "minimum": 1, "maximum": 100}
    }
  },
  "output_schema": {
    "type": "object",
    "required": ["previous", "current"],
    "properties": {
      "previous": {"type": "integer"},
      "current": {"type": "integer"}
    }
  }
}
```

Idempotência

Definir brilho em 40% duas vezes produz o mesmo estado; aumentar em 10% duas vezes não. O descriptor informa idempotência. Retries automáticos são permitidos apenas quando seguros e quando a implementação fornece request ID para deduplicação.

43. Agent Engine

Agente é um ciclo que observa estado, escolhe uma ação, recebe resultado e decide se terminou. O LLM propõe; a plataforma governa. Um limite de passos evita loops. Um orçamento limita tokens, tempo e custo. O plano é apresentado antes de tarefas longas ou de alto risco.

TEXT

```
request -> task -> plan -> next action -> policy -> tool
        -> observation -> update plan -> answer or next action
```

Estados mínimos: PLANNING, WAITING_CONFIRMATION, RUNNING_TOOL, OBSERVING, COMPLETED, CANCELLED, FAILED. Persistir estado permite recuperar UI sem repetir efeitos.

Workflow versus agente

Workflow conhece etapas. Agente escolhe dinamicamente. “Indexar este PDF” é workflow determinístico. “Prepare o ambiente para compilar este projeto” pode exigir agente, mas cada ação ainda usa tools. Prefira workflow quando a sequência é conhecida: é mais testável e previsível.

Multi-agent

Vários agentes acrescentam coordenação, estado e risco. Não são necessários ao MVP. Quando adotados, cada agente recebe capabilities mínimas e o coordenador continua sob Policy Engine comum. “Especialista em segurança” não ganha rede ou root por causa do nome.

44. Skill API e Skill Manager

Skill empacota uma capacidade reutilizável. Pode ser prompt parametrizado, workflow, tool wrapper ou agente especializado. O manifesto é legível antes da instalação.

YAML

```
schema: org.athenas.atenas.skill/1
name: summarize.document
version: 1.2.0
author: Projeto Odisseia
entrypoint: workflows/summarize.json
capabilities: [TEXT, LONG_CONTEXT]
permissions:
  - files.read:selected
risk: LOW
dependencies:
  atena: ">=0.4 <1.0"
input_schema: schemas/input.json
output_schema: schemas/output.json
tests: tests/cases.json
```

Skill Manager instala em diretório versionado, valida caminhos, tamanho, schema, dependências e assinatura quando o ecossistema a adotar. Uma Skill não executa script de instalação arbitrário. Dependências nativas vêm de pacotes Debian revisados.

Responsabilidades:

- listar e inspecionar sem executar;
- mostrar permissões antes de habilitar;
- instalar de forma atômica;
- executar testes declarados;
- habilitar, desabilitar e remover;
- manter versão anterior para rollback;
- registrar hash e origem;
- impedir alteração silenciosa de permissões em atualização.

Skill Store futura

Uma loja é distribuição e confiança, não apenas catálogo. Exige autoria, assinatura, revisão, denúncia, revogação e compatibilidade. A primeira versão usa repositório local e pacotes assinados do projeto.

45. Auditoria e observabilidade

Auditoria responde “quem pediu, qual decisão ocorreu e qual efeito foi observado?”. Log operacional responde “por que o serviço falhou?”. Métrica responde “quanto tempo e recurso consumiu?”. Não coloque tudo na mesma linha.

Registro de auditoria recomendado:

JSON

```
{
  "timestamp": "2026-08-20T12:34:56Z",
  "request_id": "018f...",
  "session_id_hash": "sha256:...",
  "provider": "llama.local",
  "model": "local-model-id",
  "tool": "system.brightness.set",
  "policy_rule": "display-change-confirm",
  "decision": "CONFIRMED",
  "result": "SUCCESS",
  "duration_ms": 184
}
```

Não registre prompt completo por padrão. Nunca registre headers de autorização, API key, conteúdo **SECRET**, áudio bruto ou arquivo. O usuário escolhe retenção. No Secure Local, auditoria

pode usar metadados mínimos ou ficar somente em memória, desde que ações críticas ainda mostrem confirmação na sessão.

IDs e correlação

Request ID acompanha Core, provider, tool e UI. Logs de componentes usam o mesmo ID, permitindo diagnóstico sem copiar conteúdo. Métricas agregadas não precisam de identidade pessoal.

46. Configuração, schema e migrações

Configuração em TOML é humana; estado dinâmico em SQLite é transacional; segredos ficam no Secret Service. Cada arquivo declara `schema_version`. O Update Engine migra um passo por vez e mantém backup antes de mudança destrutiva.

TOML

```
schema_version = 1
mode = "HYBRID"
routing_policy = "BALANCED"

[core]
max_parallel_requests = 2
default_context_tokens = 4096

[memory]
history = true
long_term = "ask"

[privacy]
ask_before_cloud = true
never_upload_files = true
```

Configuração inválida não deve iniciar com defaults perigosos. O Core aponta linha e campo, preserva o arquivo e entra em modo seguro. A UI oferece reparo explícito.

PARTE V - HARDWARE, MODELOS E PROVIDERS

47. Hardware Profiler

O Hardware Profiler transforma informações heterogêneas do Linux em um retrato versionado. Ele não depende apenas de RAM total. CPU sem extensão necessária pode não carregar um binário; GPU com VRAM suficiente pode não ter backend; armazenamento lento altera load time; bateria muda a política de threads.

Campos principais

Área	Medidas
CPU	arquitetura, modelo, núcleos, threads, frequência, topologia, ISA e cache
RAM	total, disponível, swap, zram e pressão de memória
GPU	fabricante, modelo, VRAM, driver, Vulkan, CUDA, ROCm e outros backends
Storage	filesystem, espaço livre, tipo e throughput aproximado
Energia	bateria, carregador, perfil econômico e limites térmicos
Sistema	kernel, libc, container/VM, desktop e versões dos runtimes

Fontes incluem /proc, /sys, sysinfo(2), uname(2), lscpu --json, udev e APIs de GPU quando presentes. O profiler prefere interfaces do kernel e bibliotecas, usando comandos externos como fallback parseado.

JSON

```
{
  "schema": "org.athenas.atena.hardware/1",
  "arch": "x86_64",
  "cpu": {
    "logical_threads": 2,
    "features": ["sse4_2"],
    "model": "Intel Celeron N4020"
  },
  "memory": {
    "total_bytes": 4294967296,
    "available_bytes": 2147483648,
    "zram_bytes": 1073741824
  },
  "gpu": [{"backend": "VULKAN", "usable": false}],
  "power": {"on_ac": true, "profile": "balanced"}
}
```

Perfis de produto

MINIMAL, LIGHT, STANDARD, ADVANCED e WORKSTATION são resumos para UX. O Compatibility Engine continua usando campos reais. Uma máquina pode ter 16 GB e CPU lenta; outra, 8 GB e GPU eficiente. O perfil não substitui medição.

48. Compatibility Engine

Compatibilidade possui duas perguntas: “consegue carregar?” e “oferece experiência aceitável?”. A primeira é restrição; a segunda é score.

Restrições eliminatórias:

- arquitetura do runtime não disponível;
- ISA obrigatória ausente;
- licença não aceita para o uso configurado;
- espaço insuficiente para download mais margem;
- RAM mínima de segurança não atingida;
- backend solicitado indisponível;
- formato ou arquitetura de modelo desconhecido.

Depois, um score de 0 a 100 combina memória, CPU, GPU, storage, energia e benchmark. Um exemplo inicial:

TEXT

```
score = 0.30 * memory_fit
      + 0.18 * cpu_fit
      + 0.18 * accelerator_fit
      + 0.12 * measured_speed
      + 0.08 * load_time
      + 0.07 * thermal_headroom
      + 0.07 * energy_fit
```

Pesos são dados versionados, não constantes escondidas. Antes de benchmark, `measured_speed` usa estimativa com baixa confiança. Depois do teste, a métrica real recebe maior peso. A saída inclui score, confiança e razões.

Estimativa de memória

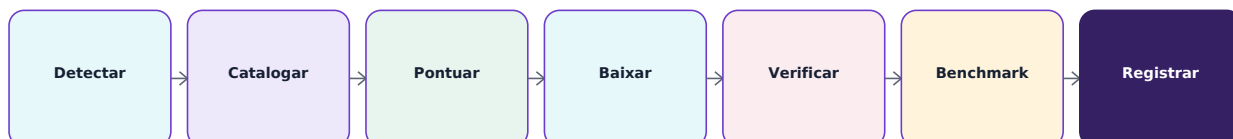
TEXT

```
required_peak = model_file_adjusted
               + kv_cache(context, layers, dimensions, cache_type)
               + runtime_buffers(batch, backend)
               + safety_margin
```

A margem preserva desktop e sistema. Em perfil LIGHT, a Atena não deve consumir toda RAM “porque existe swap”. Swap pode evitar crash, mas latência extrema destrói a experiência e aumenta desgaste.

49. Benchmark Engine

Ciclo do Model Manager



Benchmark é controlado, cancelável e reproduzível. Registra versão do runtime, hash do modelo, parâmetros e estado de energia.

Métricas:

- tempo de carregamento;
- processamento do prompt;
- time to first token (TTFT);
- tokens gerados por segundo;
- RAM de pico;
- CPU média e pico;
- uso e VRAM da GPU;
- temperatura, quando exposta;
- energia, quando mensurável;
- erros, throttling e OOM.

Execute um warm-up separado. Repita amostras curtas e guarde mediana e dispersão. Não compare números obtidos com contexto, batch e sampling diferentes como se fossem equivalentes.

JSON

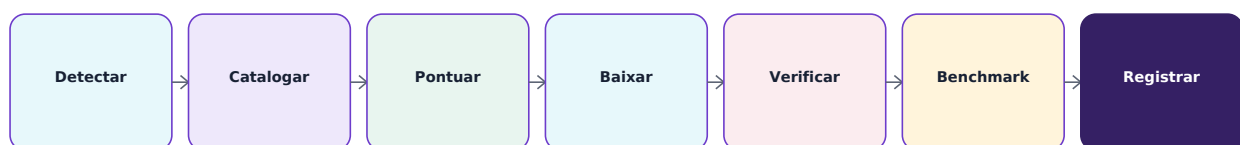
```
{
  "model_sha256": "...",
  "runtime": "llama.cpp",
  "runtime_version": "build-id",
  "context": 2048,
  "threads": 2,
  "gpu_layers": 0,
  "prompt_tokens_per_second": 18.4,
  "generation_tokens_per_second": 5.8,
  "ttft_ms": 2310,
  "peak_rss_bytes": 2469606195,
  "measured_at": "2026-08-20T12:00:00Z"
}
```

A UI pode então afirmar: “Este modelo já foi testado nesta máquina: 5,8 tokens/s e 2,3 GB de RAM no pico”. Isso é evidência local, não promessa universal.

50. Model Manager

Model Manager detecta hardware, consulta catálogo, mostra compatibilidade, baixa, verifica, instala, testa, registra, seleciona, atualiza e remove. Download ocorre em arquivo temporário, com retomada quando suportada, limite de tamanho e hash. A publicação no diretório final é atômica.

Ciclo do Model Manager



Catálogo normalizado

Campo	Por que importa
família e arquitetura	compatibilidade com runtime
parâmetros	noção de capacidade e custo
quantização e formato	memória, qualidade e loader
tamanho e hash	espaço e integridade
RAM estimada e medida	segurança de execução
contexto	limite prático configurado
capabilities	texto, visão, tools, embeddings
licença	uso, redistribuição e restrições
arquiteturas de CPU	pacote/runtime disponível
tokens/s local	experiência real

O catálogo não deve afirmar que todo GGUF é redistribuível. Modelo, tokenizer e quantização mantêm licenças próprias. A ISO principal deve evitar incluir pesos grandes ou com termos incompatíveis; o onboarding pode oferecer download pós-instalação com aceite.

Remoção segura

Antes de remover, verifique sessões ativas e referências. Mova para lixeira interna ou marque pendente; só libere bytes quando nenhum runtime usar o arquivo. Banco mantém resultado de benchmark associado ao hash, permitindo reconhecer reinstalação futura.

51. llama.cpp: runtime local de referência

llama.cpp oferece inferência em C/C++, formato GGUF, servidor HTTP e API C. Seu objetivo de execução em ampla variedade de hardware combina com o menor denominador comum do Odisseia. Isso não elimina a necessidade de testar cada arquitetura e backend.

Conceitos essenciais:

- modelo: pesos e metadados carregados do GGUF;
- contexto: estado de uma sequência ativa;
- batch: tokens submetidos ao decode;
- KV cache: estados reutilizados da atenção;
- sampling: escolha do próximo token;
- threading: paralelismo na CPU;
- offload: camadas/tensores processados em acelerador;
- backend: CPU, CUDA, ROCm, Vulkan ou outro disponível no build.

Build inicial para CPU

BASH

```
git clone https://github.com/ggml-org/llama.cpp.git
cd llama.cpp
cmake -S . -B build -DCMAKE_BUILD_TYPE=Release \
  -DGGML_NATIVE=OFF
cmake --build build -j2
```

GGML_NATIVE=OFF é útil para binário mais portátil dentro de uma arquitetura; o pacote pode oferecer variantes otimizadas e selecionar em runtime. Flags exatas evoluem, portanto o pacote fixa uma revisão testada e segue a documentação oficial daquela revisão.

Quantização e qualidade

Nomes de quantização não devem ser ordenados apenas por número de bits. Esquemas K, IQ e tipos mistos têm características específicas. O catálogo exhibe resultados medidos e não reduz a escolha a “Q4 sempre melhor para 4 GB”.

52. Fase A: Core via llama-server

Na Fase A, o runtime é um processo separado em loopback.

TEXT

```
Atena Core -> HTTP 127.0.0.1 -> llama-server -> GGUF
```

Vantagens:

- testar endpoints com ferramentas comuns;
- observar logs do runtime separadamente;
- reiniciar inferência sem derrubar o Core;
- comparar versões de llama.cpp;
- isolar crash e memória;
- usar API compatível sem criar binding C imediatamente.

INI

```
[Unit]
Description=Atena llama.cpp provider
After=aten-core.service

[Service]
ExecStart=/usr/libexec/aten/llama-server \
--host 127.0.0.1 --port 18080 \
--model %h/.local/share/aten/models/active.gguf \
--ctx-size 2048 --threads 2
Restart=on-failure
NoNewPrivileges=yes
PrivateTmp=yes
ProtectSystem=strict
ProtectHome=read-only
ReadOnlyPaths=%h/.local/share/aten/models

[Install]
WantedBy=default.target
```

O endpoint escuta somente loopback e não é exposto na LAN. O Core define timeout, limita concorrência e valida modelo ativo. Em hardware LIGHT, o service manager pode iniciar sob demanda para não reservar RAM quando a Atena não está em uso.

Adaptador HTTP

O provider converte AtenaRequest para formato aceito pelo servidor e normaliza streaming em AtenaEvent. Parâmetros não suportados retornam UNSUPPORTED; não são silenciosamente ignorados quando alteram semântica.

53. Fase B: integração com libllama

Na Fase B, um provider nativo chama a API C de llama.cpp.

TEXT

```
Atena Core -> Provider libllama -> libllama -> GGUF
```

Isso elimina HTTP e permite controle fino de contexto, batches, sampling e métricas. Também aumenta acoplamento com a versão da biblioteca, complexidade de concorrência e impacto de crash no processo. A decisão de migrar deve ser baseada em perfil e manutenção, não apenas em “menos camadas”.

Uma opção intermediária é manter `atena-llamad` como processo dedicado que usa `libllama` e protocolo Atena por socket. Assim reduz-se tradução HTTP sem colocar inferência no Core.

Ciclo de vida nativo

- 1 inicializar backends uma vez;
- 2 carregar modelo e validar metadados;
- 3 criar contexto conforme sessão/perfil;
- 4 tokenizar com limites;
- 5 executar decode em batches;
- 6 aplicar sampler configurado;
- 7 emitir tokens e métricas;
- 8 cancelar cooperativamente;
- 9 liberar contexto, modelo e backend na ordem documentada.

APIs do `llama.cpp` mudam com frequência. O pacote fixa commit/release, mantém adapter isolado e compila testes contra a versão empacotada.

54. Ollama como provider local opcional

Ollama gerencia modelos e expõe API local, simplificando a experiência em plataformas oficialmente suportadas. Na Atena, ele é um provider, não requisito.

TEXT

```
Atena -> Provider API -> Ollama -> modelo
```

O provider descobre versão, lista modelos, verifica capabilities e normaliza streaming. O Model Manager pode importar metadados, mas não deve manipular internals do diretório do Ollama. Instalação e remoção usam API/comandos documentados.

Multi-arquitetura

Documentação oficial oferece instalação Linux para amd64 e ARM64. O projeto não promete binário oficial RISC-V sem evidência. Em riscv64, `llama.cpp` é o primeiro runtime a validar; Ollama pode entrar depois por build comunitário ou suporte oficial, marcado corretamente.

Quando preferir

Ollama favorece onboarding e troca simples de modelos. `llama.cpp` direto favorece controle de build, portabilidade e perfil fino. Ambos podem coexistir; o router escolhe conforme disponibilidade e preferência.

55. Providers em nuvem e OpenAI-compatible

O Core não codifica um endpoint único. Cada provider implementa autenticação, formato, streaming, erros, usage e capabilities. Um provider `openai-compatible` recebe base URL, nomes de headers e mapeamentos configuráveis. Providers nativos podem usar recursos que um contrato compatível não cobre.

TEXT

```
providers/  
  llama/  
  ollama/  
  openai/  
  gemini/  
  anthropic/  
  deepseek/  
  mistral/  
  openai-compatible/  
  custom/
```

Adicionando um provider

- 1 implementar vtable e manifesto;
- 2 anunciar capabilities por modelo;
- 3 validar endpoint e política de rede;
- 4 obter segredo por `secret_ref`;
- 5 mapear request e streaming;
- 6 normalizar usage e erros sem vaziar conteúdo;
- 7 criar testes com servidor mock;
- 8 testar cancelamento, timeout e rate limit;
- 9 empacotar módulo e documentação;
- 10 registrar no Provider Manager.

APIs mudam

Model IDs e endpoints recomendados mudam. O livro evita hardcode de nomes “latest” no Core. Catálogos são atualizáveis e a implementação declara versão do protocolo testado. O provider mantém compatibilidade por adapter, enquanto a API interna da Atena continua estável.

56. Custo, rate limits e privacidade operacional

Provider remoto pode cobrar por tokens, requisições, áudio ou imagem. O Provider Manager registra unidade de custo e teto configurado. Antes de tarefa longa, o router estima e mostra faixa. Usage real atualiza orçamento. Ausência de preço confiável desabilita estimativa, não inventa valor.

Rate limit é tratado como estado temporário com `retry-after`, fila limitada e fallback autorizado. Não repetir tool call ou upload automaticamente. Prompts sensíveis continuam bloqueados mesmo se o provider alternativo estiver disponível.

Minimização de dados

Enviar somente trechos necessários reduz custo e exposição. Metadados de arquivo, caminho local e nome de usuário são removidos quando não necessários. Para documentos, a UI mostra quais páginas/trechos serão enviados. “Usar cloud” não é consentimento genérico eterno para qualquer arquivo.

57. Perfis por arquitetura

amd64

Plataforma de referência. Pacotes podem oferecer build baseline e variante com AVX/AVX2, selecionada por dispatcher. Nunca distribua apenas um binário compilado com `-march=native` da máquina de build.

ARM64

Segunda plataforma principal. Diferencie UEFI genérico de SBC específico. Atena é pacote de userland, mas aceleração e energia variam por placa. Teste pelo menos uma classe de desktop ARM64 e placas certificadas.

ARMv7 / armhf

Perfil Legacy/Lite. Prioriza FTS5, documentação, tools, providers remotos opcionais e modelos extremamente pequenos quando benchmark provar utilidade. Não prometa IA local pesada.

RISC-V

Canal Lab. Começa em QEMU para build e testes funcionais, depois placas selecionadas. O Debian 13 inclui riscv64 como arquitetura oficial, mas isso não uniformiza firmware, GPU nem desempenho. `llama.cpp` deve ser compilado e medido com ISA real disponível.

Arquitetura	Release Atena	Runtime local inicial	Estratégia
amd64	principal	llama.cpp + Ollama opcional	experiência completa
arm64	segunda principal	llama.cpp + Ollama quando suportado	perfis por placa
armhf	Legacy	FTS5 e provider leve	funcionalidade degradada
riscv64	Lab	llama.cpp experimental	QEMU e hardware selecionado

DECISAO ANTERIOR
Rigor de suporte

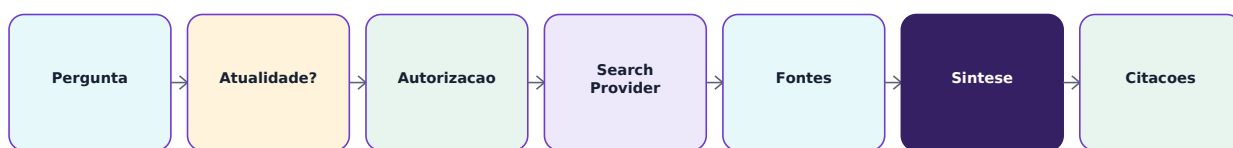
Compilar não equivale a suportar. Uma arquitetura só recebe nível de release depois de boot, serviço, provider, memória, RAG, ferramentas, atualização e testes de hardware.

PARTE VI - CONHECIMENTO, BUSCA, VOZ E EXPERIÊNCIA

58. Search Provider independente

Search Provider decide como obter informação atual, mas não pertence ao modelo. Um modelo local pode pesquisar a internet por tool autorizada; um modelo remoto pode responder sem busca; localidade do modelo e conectividade são eixos diferentes.

Pesquisa com rastreabilidade



O Task Classifier identifica sinais de atualidade, citação ou desconhecimento. Privacy Engine verifica se a consulta pode sair da máquina. Search Provider recebe uma consulta minimizada, busca resultados, coleta fontes dentro de limites e devolve documentos normalizados. O modelo sintetiza e Citation Engine liga afirmações aos locators.

C

```
typedef struct {
    const char *query;
    size_t max_results;
    bool current_information_required;
    const char *language;
    const char **allowed_domains;
    size_t allowed_domain_count;
} AtenaSearchRequest;

typedef struct {
    const char *title;
    const char *url;
    const char *snippet;
    const char *published_at;
    double relevance;
} AtenaSearchResult;
```

Proteções

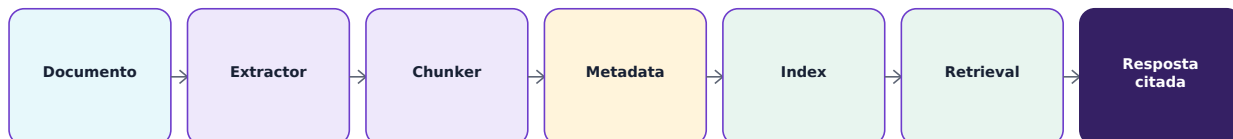
Fetcher limita esquemas a HTTPS/HTTP permitido, bloqueia loopback e redes privadas por padrão para evitar SSRF, limita redirects, bytes, MIME e tempo. HTML é convertido em texto sem executar JavaScript. Downloads viram anexos separados e passam por política. Robots, termos de uso e direitos autorais são respeitados.

Fontes e atualidade

Resultados de busca não são evidência suficiente. A coleta abre fontes e registra data de publicação e de acesso quando disponíveis. Para documentação técnica, fontes oficiais e repositórios primários têm prioridade. A resposta distingue fato da fonte e inferência da Atena.

59. RAG: recuperação antes de geração

Pipeline da base de conhecimento



RAG adiciona memória não paramétrica. O pipeline ingere documento, extrai texto, divide em chunks, associa metadados, indexa, recupera e entrega evidências ao modelo. Uma resposta confiável preserva documento, página, trecho, arquivo ou URL.

Ingestão

O extractor detecta MIME por conteúdo, não só extensão. PDF textual usa biblioteca de extração; PDF escaneado requer OCR opcional e marca confiança. Código preserva arquivo e linhas. Markdown preserva headings. Arquivos protegidos ou grandes exigem consentimento e limite.

Chunking

Chunks não devem cortar qualquer quantidade fixa de caracteres. Estratégia hierárquica usa seções, parágrafos e tamanho máximo em tokens, com pequena sobreposição. Cada chunk mantém `document_id`, página/linha, heading, hash e privacy class.

JSON

```
{
  "chunk_id": "sha256:...#p21-c03",
  "document_id": "project-architecture",
  "source_name": "Projeto_Odisseia_Athenas_OS_Arquitetura_Tecnica.pdf",
  "page_start": 21,
  "page_end": 21,
  "heading": "IA Atena",
  "text": "...",
  "privacy": "PRIVATE",
  "extractor_version": "pdf-text/1"
}
```

60. RAG Lite com SQLite FTS5

FTS5 oferece busca de texto completo sem embeddings. É pequeno, auditável e adequado a hardware MINIMAL/LIGHT. A busca lexical encontra termos, prefixos, frases e proximidade. BM25 ordena relevância.

SQL

```

CREATE TABLE documents (
  id TEXT PRIMARY KEY,
  path TEXT NOT NULL,
  title TEXT,
  sha256 TEXT NOT NULL,
  privacy_class TEXT NOT NULL,
  indexed_at TEXT NOT NULL
);

CREATE TABLE chunks (
  rowid INTEGER PRIMARY KEY,
  id TEXT UNIQUE NOT NULL,
  document_id TEXT NOT NULL REFERENCES documents(id) ON DELETE CASCADE,
  page_start INTEGER,
  page_end INTEGER,
  heading TEXT,
  body TEXT NOT NULL
);

CREATE VIRTUAL TABLE chunks_fts USING fts5(
  heading,
  body,
  content='chunks',
  content_rowid='rowid',
  tokenize='unicode61 remove_diacritics 2'
);

```

Sincronize external content com triggers ou rotina transacional documentada. Consultas do usuário nunca entram como SQL bruto; use `MATCH ?` com parser que limita operadores.

Expansão de consulta

Um modelo pequeno pode sugerir sinônimos, mas o sistema preserva consulta original e mostra expansão. Em português, remoção de diacríticos ajuda “configuracao” encontrar “configuração”, mas stemming e termos técnicos precisam de avaliação.

61. RAG Standard e Advanced

RAG Standard adiciona embeddings locais. Um modelo de embedding transforma consulta e chunks em vetores. Similaridade por cosseno recupera sem depender de palavras idênticas. O modelo de embedding é separado do gerador e aparece no Model Manager com licença, dimensão e benchmark.

RAG Advanced combina busca lexical e vetorial, funde rankings e aplica reranker local quando hardware permite.

TEXT

```

lexical candidates ----+
                        +-> reciprocal rank fusion -> reranker -> top evidence
vector candidates -----+

```

Reciprocal Rank Fusion combina posições sem exigir scores na mesma escala. Reranking melhora ordem, mas acrescenta latência. O perfil LIGHT pode usar apenas FTS5; STANDARD usa híbrido sem reranker; ADVANCED habilita o pipeline completo.

Avaliação

Crie perguntas com documentos e locators esperados. Meça `Recall@k`, precisão, MRR e cobertura de citação. Em seguida, avalie se a resposta usa somente evidências. Um retrieval excelente ainda pode ser mal sintetizado; as duas fases têm testes separados.

62. Citation Engine

Citação não é um link decorativo. Cada trecho de evidência recebe ID e locator. O modelo emite referências estruturadas, que o Core valida. Se uma referência não pertence ao conjunto recuperado, é rejeitada.

JSON

```
{
  "answer_blocks": [
    {
      "text": "A Atena foi definida como socrática, híbrida e regional.",
      "citations": ["evidence-2"]
    }
  ],
  "evidence": {
    "evidence-2": {
      "document": "Projeto Odisseia.pdf",
      "page": 8,
      "locator": "p8"
    }
  }
}
```

Para URL, guarde URL canônica, título, data de acesso e data do conteúdo quando conhecida. Para código, arquivo, commit e intervalo de linhas. Para PDF, página impressa e índice interno podem diferir; a UI mostra ambos quando necessário.

Resposta insuficiente

Se a evidência não sustenta a afirmação, a Atena deve dizer que não encontrou base. RAG não transforma qualquer resultado em verdade. Uma opção “somente fontes” proíbe conhecimento externo do modelo para tarefas de auditoria.

63. Personal Knowledge Base

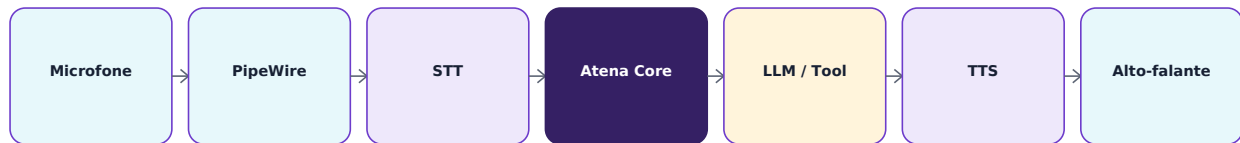
O usuário adiciona PDFs, livros, apostilas, Markdown, código, manuais, notas e diretórios. Cada coleção possui nome, política, idioma, extractor e modo de atualização. Diretórios são observados com limites; arquivos temporários, binários e segredos seguem ignore rules.

Coleção	Exemplo	Política sugerida
Documentação Athenas	manuais e decisões	pública/local, versionada
Estudos	apostilas e notas	privada, exportável
Projetos	código e README	regra por diretório
Regional	conteúdo educacional revisado	citável e versionado
Security Lab	amostras e relatórios	Secure Local obrigatório

O usuário pode pausar indexação, remover documento, reconstruir índice e exportar metadados. Remover fonte apaga chunks e embeddings. Cache de OCR é tratado como cópia do documento e segue a mesma privacidade.

64. Assistente de voz opcional

Fluxo de voz opcional



Voz é uma interface, não um caminho privilegiado. A transcrição vira request comum; tools seguem a mesma política. A resposta textual pode ser sintetizada por TTS. O usuário pode cancelar falando, pressionando tecla ou clicando.

Captura com PipeWire

PipeWire fornece captura e reprodução com modelo de segurança apropriado ao desktop. A primeira implementação pode usar `pw-record`/biblioteca de alto nível; a versão nativa usa `pw_stream`. A UI mostra indicador de microfone e dispositivo ativo.

STT

Opções locais devem ser avaliadas por idioma e hardware. `whisper.cpp` oferece caminho C/C++; Vosk possui modelos pequenos e português; serviços remotos são fallback opcional. O Provider Manager trata STT como capability, com latência, memória, licença e rede.

TTS

Piper é referência de TTS local rápido, mas versão, licença e API precisam ser verificadas no empacotamento. Speech Dispatcher pode integrar acessibilidade. Vozes são modelos separados com idioma, licença, tamanho e amostras.

Wake word

Wake word é futura e desabilitada por padrão. Escuta contínua afeta privacidade, bateria e falsos positivos. O detector local não transmite áudio; a UI fornece chave física/lógica para desligar. Buffer atrasado é curto, volátil e documentado.

Streaming e interrupção

STT parcial pode atualizar a UI sem disparar ação até confirmação de final de fala. TTS começa por sentenças, mas interrompe imediatamente quando usuário cancela. Echo cancellation evita que a própria voz da Atena volte ao microfone.

65. Atena CLI

CLI é interface de primeira classe para diagnóstico, automação e acessibilidade.

BASH

```
atena ask "Explique ponteiros com um exemplo"
atena ask --offline --cite @manual-c.pdf "O que é malloc?"
atena models list
atena models benchmark local-model-id
atena providers doctor
atena skills inspect summarize.document
atena memory export --category preferences
atena privacy status
```

Saída humana vai a stdout; erros a stderr; `--json` produz schema estável. Códigos de saída distinguem uso inválido, indisponibilidade, bloqueio de política e falha de provider. Segredos não são aceitos na linha de comando.

Integração com comandos Athenas

O comando anteriormente proposto `athenas-ai` pode ser um alias de compatibilidade que chama `atena`. `athenas-hw` compartilha o Hardware Profiler; `athenas-health` inclui saúde do Core e providers. Evite duplicar parsers e lógica.

66. Atena UI, sidebar e overlay

Uma UI própria pode ser construída inicialmente com toolkit disponível no Athenas OS, mantendo o Core independente. A tela principal mostra conversa, fontes, provider/modelo, modo, uso de ferramentas e cancelamento. Model Manager, Provider Manager, Skill Manager e privacidade aparecem em painéis separados.

Entradas

- aplicativo completo;
- `Super + Space` para overlay;
- sidebar acoplável;
- ícone/taskbar;
- menu de seleção de texto;
- alvo para arrastar arquivo;
- terminal;
- voz opcional.

Seleção de texto

O serviço oferece Explicar, Resumir, Traduzir, Pesquisar, Estudar, Reescrever e Perguntar à Atena. A seleção não é enviada até a ação ser escolhida. Aplicações sob Wayland podem exigir protocolos/portais e integração específica; a arquitetura não promete captura global insegura.

Arrastar arquivo

O drop target recebe URI, consulta MIME e exibe ações compatíveis: resumir, indexar, perguntar, comparar, extrair. Diretório ou arquivo especial é rejeitado. Antes de cloud, a UI mostra o que será transmitido.

Acessibilidade

Toda função essencial é navegável por teclado, rotulada para AT-SPI/Orca, operável com escala aumentada e não depende apenas de cor. Streaming respeita leitor de tela: atualizações são agrupadas para não interromper continuamente. Animações podem ser reduzidas. Voz não substitui acessibilidade textual.

67. Model, Provider, Agent e Skill Managers na interface

Gerenciadores não devem parecer uma única lista de “IAs”. O Model Manager mostra artefatos e benchmark. Provider Manager mostra runtimes/endpoints e segredos referenciados. Agent Manager mostra objetivos e workflows. Skill Manager mostra capacidades e permissões.

Uma ficha de modelo inclui:

Campo	Exibição
Modelo/família	nome normalizado e origem
Parâmetros	total ou arquitetura MoE quando aplicável
Quantização/formato	tipo e GGUF/alternativo
Tamanho	download e instalado
RAM	estimada e pico medido
Contexto	suportado e configurado
Velocidade	prompt, TTFT e geração
Capabilities	badges textuais acessíveis
Licença	nome e link/arquivo
Compatibilidade	score, confiança e razões

Instalar modelo é operação explícita. A UI verifica hash e mostra origem. Atualização que muda pesos recebe novo hash e benchmark separado.

68. Experiência pedagógica e transparência

O usuário pode abrir “Como esta resposta foi produzida” e ver, em linguagem simples:

- modo ativo;
- provider e modelo;
- se houve internet;
- documentos consultados;
- tools propostas/executadas;
- confirmações;
- tempo e uso aproximado;
- itens que não foram enviados por privacidade.

Transparência não significa expor raciocínio privado do modelo. Significa revelar decisões de sistema, fontes e efeitos verificáveis.

Modo estudo

No modo estudo, a Atena pergunta objetivo e nível, divide tarefa, propõe exercício e verifica compreensão. O usuário pode alternar para resposta direta. Progresso persistente é opt-in. A UI distingue pista, explicação, solução e revisão.

69. ATENA SECURITY LAB

O perfil Security Lab prioriza execução local, workspace isolado e ferramentas controladas. Serve a aprendizagem de Linux, redes, secure coding, CTF, pentest autorizado, blue team, análise de vulnerabilidade, engenharia reversa, fuzzing, debugging, PoC controlada e análise de malware.

TEXT

```
Security Lab -> Atena local -> workspace isolado
              -> containers/VMs/namespaces -> tools allowlisted
              -> rede de laboratorio -> auditoria
```

Quatro contextos que não podem ser confundidos

Contexto	Exemplo	Regra
Estudo de código ofensivo	ler exploit público	ambiente local, análise e ética
Análise de malware	observar amostra	VM sem confiança, snapshots e rede controlada
PoC em laboratório	reproduzir falha em alvo próprio	escopo e isolamento explícitos
Ação contra sistema real	testar host de terceiro	autorização verificável e ferramentas governadas

A IA não ganha shell irrestrito. Tools de laboratório recebem alvos permitidos, faixa de rede isolada e limites. Uma tool `security.lab.scan` valida que o alvo pertence ao workspace/allowlist. A distro principal não precisa incluir arsenal de pentest; o módulo Lab é separado.

**LIMITE DE
SEGURANÇA**
**A autorização é
propriedade do
contexto**

Uma frase como “é para estudo” não altera tecnicamente o alvo. O perfil exige ambiente controlado e ainda bloqueia ações fora do escopo definido.

Malware

Arquivos suspeitos não entram no RAG geral, não são enviados à nuvem e não são abertos por aplicativos comuns. Extração ocorre em VM/container apropriado, com hashes, cópia somente leitura e logs. Containers não são fronteira suficiente contra toda classe de malware; VM é preferível para amostras hostis.

PARTE VII - EMPACOTAR, INTEGRAR, TESTAR E LANÇAR

70. Pacotes Debian da Atena

Atena não deve ser copiada por hook como um conjunto de arquivos sem identidade. Componentes viram pacotes `.deb`, com versão, dependências, arquitetura, conffiles e ciclo de atualização.

Divisão inicial:

Pacote	Arquitetura	Conteúdo
atena-core	any	serviço, biblioteca e schemas
atena-cli	any	cliente terminal
atena-ui	any ou all	interface, conforme toolkit
atena-rag	any	SQLite/extractors e serviço de indexação
atena-voice	any	captura e adapters STT/TTS
atena-provider-llama	any	adapter e runtime/pacote dependente
atena-provider-ollama	any/all	adapter opcional
atena-provider-cloud	any	adapters HTTP comuns
atena-data	all	policy packs, prompts, schemas e docs
atena-security-lab	any/all	perfil opcional e manifests
athenas-ai	all	metapacote que seleciona conjunto padrão

Architecture: `any` é recompilado para cada arquitetura; `all` contém dados independentes. Um metapacote não possui implementação: declara dependências da edição.

Arquivo debian/control

```
DEBCONTROL

Source: atena
Section: utils
Priority: optional
Maintainer: Projeto Odisseia <maintainers@example.invalid>
Build-Depends: debhelper-compat (= 13), cmake, pkg-config,
  libjson-c-dev, libsqlite3-dev, libcurl4-openssl-dev,
  libsystemd-dev
Standards-Version: 4.7.2
Rules-Requires-Root: no

Package: atena-core
Architecture: any
Depends: ${shlibs:Depends}, ${misc:Depends}
Description: Core da plataforma de inteligencia artificial Atena
  Servico local, roteamento, politica, memoria e contratos da Atena.

Package: atena-cli
Architecture: any
Depends: ${shlibs:Depends}, ${misc:Depends}, atena-core (= ${binary:Version})
Description: Cliente de terminal da plataforma Atena
  Interface de linha de comando e diagnostico.
```

Use endereço real antes da publicação. Descrições não prometem recursos ausentes. Licenças de código, modelos, vozes e assets entram em `debian/copyright`.

Build do pacote

BASH

```
sudo apt build-dep ./
dpkg-buildpackage -us -uc -b
lintian ../atena_*.changes
```

Na CI, use `sbuild` ou `pbuilder` para ambiente limpo. `-us -uc` cria pacote não assinado de desenvolvimento; `release` usa assinatura do processo oficial.

71. Serviços, D-Bus, Polkit e desktop no pacote

Arquivos são instalados em locais previsíveis:

Artefato	Destino
serviço de usuário	<code>/usr/lib/systemd/user/atena-core.service</code>
ativação D-Bus	<code>/usr/share/dbus-1/services/...</code>
interface D-Bus	<code>/usr/share/dbus-1/interfaces/...xml</code>
ação Polkit	<code>/usr/share/polkit-1/actions/...policy</code>
executáveis internos	<code>/usr/libexec/atena/</code>
schemas	<code>/usr/share/atena/schemas/</code>
desktop entry	<code>/usr/share/applications/atena.desktop</code>
ícones	<code>/usr/share/icons/hicolor/.../apps/atena.*</code>
documentação	<code>/usr/share/doc/atena-*</code>

Não habilite serviço global root. Serviço de usuário pode ser ativado por D-Bus ou socket sob demanda. Maintainer scripts evitam alterar `$HOME`; configuração por usuário nasce no primeiro uso.

Exemplo de ação Polkit

XML

```
<?xml version="1.0" encoding="UTF-8"?>
<!DOCTYPE policyconfig PUBLIC
"-//freedesktop//DTD PolicyKit Policy Configuration 1.0//EN"
"http://www.freedesktop.org/standards/PolicyKit/1/policyconfig.dtd">
<policyconfig>
  <action id="org.athenas.atena.power.schedule-shutdown">
    <description>Agendar desligamento solicitado pela Atena</description>
    <message>Autentique-se para agendar o desligamento</message>
    <defaults>
      <allow_any>no</allow_any>
      <allow_inactive>no</allow_inactive>
      <allow_active>auth_admin_keep</allow_active>
    </defaults>
  </action>
</policyconfig>
```

O helper oferece apenas `schedule-shutdown` com intervalo validado. A tool de cancelamento é separada e auditável.

72. Integrando à build do Athenas OS

Da fonte ao Athenas OS



O script atual gera arquivos por heredoc e usa Bookworm/amd64. A evolução recomendada mantém o script como orquestrador e move configurações ao repositório. A integração da Atena acontece pela lista de pacotes e repositório local/assinado, não pela cópia manual de binários.

Durante desenvolvimento

- 1 construir pacotes em ambiente limpo;
- 2 criar repositório APT local de staging;
- 3 adicionar chave de desenvolvimento à imagem de teste;
- 4 incluir `athenas-ai` somente na edição escolhida;
- 5 construir ISO;
- 6 bootar em QEMU e testar serviço;
- 7 guardar manifest, hashes e logs.

Exemplo de package list da edição:

TEXT

```
atena-core  
atena-cli  
atena-ui  
atena-rag  
atena-data  
atena-provider-llama
```

Voice, Ollama e Security Lab são opcionais. Pesos GGUF não entram por padrão. O onboarding detecta hardware e apresenta modelos compatíveis.

Bookworm e Trixie

Enquanto o script corrigido estiver em Bookworm, mantenha branch/pipeline de compatibilidade. O release alvo da nova arquitetura usa Debian 13 Trixie. Nunca misture repositórios Bookworm e Trixie para “resolver” dependências.

ISO não é a unidade de atualização

Depois da instalação, Atena é atualizada por APT. Modelos e Skills usam managers com catálogos próprios, hashes e políticas, mas executáveis continuam preferencialmente em pacotes assinados. Não reconstrua a ISO para corrigir uma tool.

73. Update Engine e compatibilidade de dados

Update Engine coordena versão do Core, schema de banco, configuração, providers e Skills. Antes de iniciar binário novo, o pacote preserva dados. Na primeira execução, o Core verifica compatibilidade e aplica migrações transacionais.

Regras:

- nunca migrar banco sem backup consistente;
- cada migração possui `up`, teste e estratégia de falha;
- rollback de pacote não presume downgrade automático de schema;
- providers declaram faixa de ABI;
- Skills que pedem novas permissões ficam desabilitadas até revisão;
- catálogos são assinados e possuem validade;
- logs de migração não contêm dados do usuário.

Separar atualização de modelo

“Atualizar modelo” pode mudar qualidade, licença e memória. O Model Manager baixa nova versão lado a lado, verifica, executa smoke benchmark e permite retorno ao hash anterior. Alias `active` muda atomicamente.

74. Estratégia completa de testes

Testar o chatbot visualmente não prova a plataforma. A matriz combina camadas.

Camada	Testes
C unitário	parsers, buffers, score, regras e lifecycle
Contrato	schemas de provider, tool, Skill e eventos
Integração	mock HTTP, SQLite, D-Bus e systemd user
E2E	CLI/UI -> Core -> provider -> resposta/citação/tool
Segurança	fuzz, injection, SSRF, path traversal, segredo em log
Desempenho	load, TTFT, tokens/s, RAM, indexação e startup
Acessibilidade	teclado, Orca, contraste, escala e redução de movimento
Empacotamento	install, upgrade, purge, conffile e licença
Arquitetura	build, boot e hardware real por tier

Provider mock

Um servidor local simula streaming, timeout, erro 429, JSON inválido e disconnect. Testes não dependem de API comercial. Golden tests comparam eventos normalizados, não texto criativo completo.

Policy tests

Toda combinação crítica vira tabela: `PRIVATE + cloud = DENY`; `SENSITIVE + cloud + sem confirmação = CONFIRM`; `SECRET = nunca prompt`. Alterar regra exige atualizar caso e revisão.

Tool tests

Use doubles para D-Bus e filesystem. O teste verifica que argumentos inválidos nunca chegam à implementação, confirmação ocorre antes do efeito e auditoria não contém segredo.

75. Testes de segurança ofensiva autorizada da própria Atena

O projeto deve tentar quebrar suas fronteiras em laboratório.

Casos:

- documento RAG ordena ignorar política;
- modelo inventa nome de tool;
- chamada possui campo extra e tipo errado;
- symlink muda entre aprovação e uso;
- URL redireciona para 127.0.0.1 ou metadata service;
- provider devolve chunk maior que limite;
- Skill contém . / no pacote;
- log de erro inclui header Bearer;
- cancelamento ocorre durante tool não idempotente;
- arquivo privado é anexado a request cloud;
- usuário sem sessão ativa tenta helper Polkit.

Cada falha gera regression test. Security Lab não é o único contexto de segurança; o próprio Core é superfície de ataque.

76. Evals pedagógicas e de confiabilidade

Para o método socrático, avalie se a Atena detecta nível sem humilhar, oferece pista progressiva, não entrega solução cedo demais quando modo estudo está ativo e aceita pedido de resposta direta. Educadores revisam amostras. Métricas automáticas ajudam, mas não substituem julgamento.

Para confiabilidade, crie tarefas com fonte conhecida e verifique:

- afirmação apoiada por evidência;
- citação aponta ao trecho correto;
- incerteza aparece quando fontes divergem;
- atualidade aciona busca somente quando autorizada;
- provider e modelo são informados corretamente;
- memória não inventa preferência.

77. CI, release e cadeia de suprimentos

Pipeline por commit:

- 1 formatação e análise estática;
- 2 build GCC e Clang;
- 3 unit tests e sanitizers;
- 4 schema tests e fuzz corpus curto;
- 5 pacote Debian em ambiente limpo;
- 6 Lintian;
- 7 integração em container/VM;
- 8 SBOM e hashes;
- 9 assinatura somente em job protegido;
- 10 publicação no canal correto.

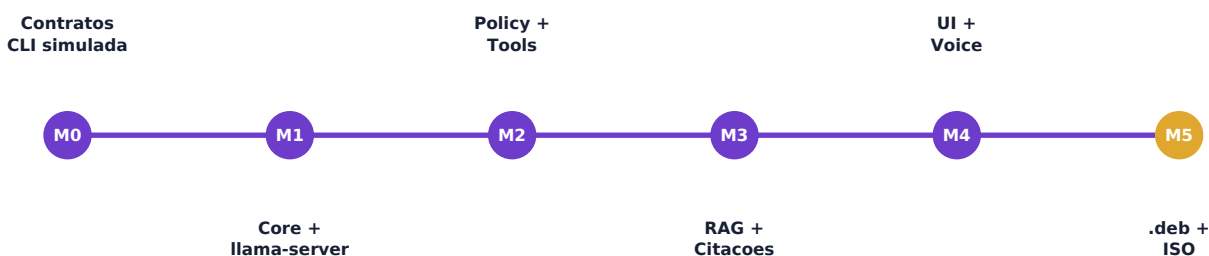
Dependências são fixadas por versão/revisão testada e atualizadas conscientemente. Binários baixados no build exigem hash e origem; preferencialmente use source package Debian. Segredos de assinatura nunca entram na CI de pull request.

Canais

dev aceita mudanças rápidas. testing recebe builds que passaram automação. stable exige matriz de hardware e documentação. security entrega correções prioritárias. O canal RISC-V Lab não é anunciado como stable.

78. Roadmap da Atena

Roadmap verificavel da Atena



M0 - Contratos e simulador

Headers em C, schemas, CLI, provider mock, SQLite inicial e testes. Nenhuma inferência real. Critério: requests e eventos atravessam pipeline e política é testável.

M1 - Inferência local

Provider llama-server, Model Manager mínimo, profiler e benchmark. Critério: modelo GGUF compatível é instalado, verificado, testado e usado offline.

M2 - Tools e permissões

Tool Engine, Policy Engine, confirmação, D-Bus e três tools de baixo/médio risco. Critério: nenhuma ação usa shell gerado; auditoria é completa e sem segredos.

M3 - RAG e fontes

FTS5, ingestão de PDF/Markdown/código, Citation Engine e modo somente fontes. Critério: conjunto de perguntas retorna locators corretos.

M4 - UI e voz

Janela, overlay, seleção de texto, drop de arquivo e voz opcional. Critério: acessibilidade por teclado/Orca e cancelamento de streaming.

M5 - Pacotes e ISO

Pacotes .deb, repo de testing, service activation, onboarding e integração Athenas OS. Critério: instalação limpa, upgrade e remoção em VM e hardware.

M6 - ARM64 e RISC-V Lab

Cross-build, QEMU e placas selecionadas. Critério: matriz publicada com limites e benchmark real.

79. Projeto final: uma Atena mínima utilizável

O capstone reúne o livro em uma entrega pequena, mas vertical.

Requisitos

- `atena-core` como serviço de usuário;
- `atena-cli ask` com streaming;
- `provider mock` e `llama-server`;
- modos Offline e Local conectado;
- Privacy Engine com cinco classes;
- FTS5 para documentos Markdown/PDF textual;
- citação por arquivo/página;
- `tool system.info.read` sem privilégio;
- `tool system.brightness.set` com confirmação;
- SQLite para conversas e preferências;
- Model Manager capaz de registrar e benchmarkar um GGUF;
- pacote `.deb` e teste em ISO.

Sequência de implementação

- 1 criar repositório e CMake;
- 2 implementar status, buffer e request;
- 3 implementar event sink e CLI;
- 4 criar provider mock;
- 5 adicionar policy e testes;
- 6 integrar SQLite;
- 7 integrar `llama-server`;
- 8 adicionar profiler/benchmark;
- 9 implementar FTS5 e citações;
- 10 implementar tools;
- 11 mover Core a `systemd` user + IPC;
- 12 empacotar e integrar à imagem.

Ao terminar, a pessoa terá construído uma plataforma de agente pequena, não um “modelo próprio”. Essa distinção é a maturidade central do projeto.

PARTE VIII - IMPLEMENTAÇÃO DE REFERÊNCIA E CONTRATOS

80. Esqueleto de diretórios compilável

Caminho	Arquivo inicial
include/atena/status.h	enum de erros e mensagens
include/atena/request.h	request imutável e builder
include/atena/provider.h	vtable, capability e eventos
include/atena/policy.h	classificação e decisão
core/request.c	validação e cópia segura
core/router.c	filtros e score
core/policy.c	regras determinísticas
providers/mock.c	provider de teste
providers/llama_http.c	adapter Phase A
memory/store.c	SQLite e migrações
rag/fts.c	FTS5 e locators
tools/registry.c	registro e schemas
cli/main.c	parser CLI e event rendering
tests/	unidade e integração

81. Provider mock

O provider mock é o primeiro motor porque torna arquitetura testável sem download, GPU ou rede.

```
c
```

```
typedef struct {
    unsigned delay_ms;
    bool fail_after_first_chunk;
} MockState;

static AtenaStatus mock_generate(
    AtenaProvider *self,
    const AtenaRequest *request,
    AtenaStreamSink *sink
) {
    if (self == NULL || request == NULL || sink == NULL) {
        return ATENA_ERR_INVALID;
    }

    const char *chunks[] = {
        "Atena recebeu: ",
        request->user_text,
        "\nResposta gerada pelo provider de teste."
    };

    MockState *state = self->impl;
    for (size_t i = 0; i < 3; ++i) {
        AtenaEvent event = {
            .type = ATENA_EVENT_TEXT_DELTA,
            .request_id = request->id,
            .json_payload = chunks[i]
        };
        AtenaStatus status = sink->emit(sink, &event);
        if (status != ATENA_OK) return status;
        if (state->fail_after_first_chunk && i == 0) {
            return ATENA_ERR_PROVIDER;
        }
    }
    return sink->done(sink, request->id);
}
```

O teste simula erro após chunk, cancelamento e sink que recusa escrita. A UI precisa terminar estado corretamente em todos.

82. Router determinístico inicial

C

```
AtenaStatus select_provider(
    const AtenaRequest *request,
    const AtenaProviderInfo *providers,
    size_t count,
    size_t *selected
) {
    if (request == NULL || providers == NULL || selected == NULL) {
        return ATENA_ERR_INVALID;
    }

    double best = -1.0;
    size_t best_index = 0;
    bool found = false;

    for (size_t i = 0; i < count; ++i) {
        const AtenaProviderInfo *p = &providers[i];
        if (!p->is_available) continue;
        if ((p->capabilities & request->required_capabilities)
            != request->required_capabilities) continue;
        if (!request->allow_network && !p->is_local) continue;
        if ((request->privacy == ATENA_PRIVACY_PRIVATE ||
            request->privacy == ATENA_PRIVACY_SECRET) && !p->is_local) continue;

        double score = p->is_local ? 10.0 : 5.0;
        score += p->health_score;
        score += p->quality_score;
        score -= p->estimated_cost_score;

        if (!found || score > best) {
            found = true;
            best = score;
            best_index = i;
        }
    }

    if (!found) return ATENA_ERR_UNSUPPORTED;
    *selected = best_index;
    return ATENA_OK;
}
```

Produção também retorna justificativas. Este código demonstra a ordem: filtros de segurança antes do score.

83. Registro de tools

C

```
typedef AtenaStatus (*AtenaToolRun)(
    const char *arguments_json,
    AtenaToolContext *context,
    char **result_json
);

typedef struct {
    const char *name;
    uint32_t version;
    AtenaRisk risk;
    bool idempotent;
    const char *input_schema_path;
    const char *output_schema_path;
    AtenaToolRun run;
} AtenaToolDescriptor;

AtenaStatus tool_registry_add(
    AtenaToolRegistry *registry,
    const AtenaToolDescriptor *descriptor
);
```

Registry rejeita nome duplicado, versão zero, schema ausente ou função nula. Tool Engine copia descriptor ou exige lifetime estático documentado. Plugins não podem sobrescrever tool do sistema.

84. Migração SQLite

C

```
static const char *MIGRATION_001 =
    "BEGIN IMMEDIATE;"
    "CREATE TABLE IF NOT EXISTS schema_migrations("
    "  version INTEGER PRIMARY KEY, applied_at TEXT NOT NULL);"
    "CREATE TABLE IF NOT EXISTS conversations("
    "  id TEXT PRIMARY KEY, title TEXT, created_at TEXT NOT NULL,"
    "  privacy_class TEXT NOT NULL, retention_policy TEXT NOT NULL);"
    "INSERT INTO schema_migrations(version, applied_at)"
    "  VALUES(1, datetime('now'));"
    "COMMIT;"
```

Se qualquer comando falhar, execute `ROLLBACK`, registre apenas código/etapa e preserve backup. Migrações reais são arquivos numerados para revisão, não strings enormes no Core.

85. Contrato de Skill completo

Campos obrigatórios:

Campo	Validação
schema	URI e versão suportada
name	namespace ASCII estável
version	SemVer
author	identidade textual e opcional chave
entrypoint	caminho relativo sem traversal
permissions	valores conhecidos e mínimos
input_schema	JSON Schema local validado
output_schema	JSON Schema local validado
capabilities	conjunto suportado
risk	nível conhecido
dependencies	ranges limitados e resolvíveis
tests	casos sem rede por padrão

Uma atualização que adiciona `files.write` quando a versão anterior tinha apenas `files.read:selected` não é habilitada automaticamente.

86. Checklist de Definition of Done

Uma feature Atena está pronta quando:

- possui requisito e ameaça descritos;
- contrato e schema estão versionados;
- implementação compila com GCC e Clang;
- erros e cancelamento são tratados;
- teste unitário e de integração passam;
- dados e logs foram classificados;
- acessibilidade foi testada quando há UI;
- pacote instala e remove limpo;

- documentação explica limite e fallback;
- benchmark existe quando desempenho importa;
- matriz de arquitetura registra o estado real;
- nenhuma chave, peso ou licença foi incorporada indevidamente.

PARTE IX - REFERÊNCIAS, GLOSSÁRIO E PRÓXIMOS PASSOS

87. Glossário essencial

Termo	Definição no projeto
ABI	contrato binário entre componentes compilados
Agent	ciclo que seleciona ações sob política
API	contrato de comunicação entre módulos
Backend	implementação de computação, como CPU/Vulkan/CUDA
Capability	capacidade anunciada por provider/model/tool
Context	informação temporária fornecida ao modelo
Embedding	vetor que representa conteúdo para comparação
FTS5	módulo de busca textual do SQLite
GGUF	formato de modelo e metadados usado pelo ecossistema GGML
IPC	comunicação entre processos
KV cache	estados de atenção reutilizados durante geração
LLM	modelo de linguagem de grande escala
Locator	posição verificável em uma fonte
LoRA	adaptação de baixa dimensão sobre modelo congelado
Polkit	autorização de ações privilegiadas no desktop Linux
Provider	adaptador para runtime ou serviço cognitivo
RAG	geração apoiada por recuperação de fontes
Schema	regra formal de estrutura e tipos de dados
Skill	pacote versionado de capacidade reutilizável
STT	conversão de voz para texto
Tool	função tipada que observa ou altera estado
TTS	conversão de texto para voz

88. Fontes primárias do Projeto Odisseia

- Projeto Odisseia.pdf, 12 páginas: missão, desafios, pilares, IA Atena, identidade regional, OdysseyOS e narrativa educacional.
- Slide.pdf, 12 páginas: história do cyberdeck, cyberpunk, cultura maker/DIY, processamento, interface, energia, conectividade, Linux e cibersegurança autorizada.
- Projeto_Odisseia_Athenas_OS_Arquitetura_Tecnica.pdf, 42 páginas: arquitetura em camadas, Debian 13, pacotes, segurança, Atena, multi-arquitetura e roadmap.
- criar-athenas-linux-corrigido.sh: estado verificável do construtor Debian Live Bookworm/amd64 e branding.

89. Referências oficiais de integração com IA no sistema

- Deepin, “UOS AI 2.0 is released”: deepin.org/en/uos-ai-2-0-released.
- Deepin 25 Release Note: deepin.org/en/deepin-25-release.
- Deepin 25 e UOS AI Skills: deepin.org/en/deepin-25-uos-ai.
- Deepin 25.1 announcement: deepin.org/en/deepin-25-1-announcement.

Consultadas em agosto de 2026. Recursos do UOS AI são estudo de caso e não declaração de implementação na Atena.

90. Referências oficiais da pilha técnica

- Debian 13 Trixie: debian.org/releases/trixie.
- Debian 13 Installation Information: debian.org/releases/trixie/debian-installer.
- Debian Live Manual: live-team.pages.debian.net/live-manual.
- llama.cpp: github.com/ggml-org/llama.cpp.
- Build do llama.cpp: github.com/ggml-org/llama.cpp/blob/master/docs/build.md.
- Servidor HTTP do llama.cpp: github.com/ggml-org/llama.cpp/blob/master/tools/server/README.md.
- Ollama para Linux: docs.ollama.com/linux.
- SQLite FTS5: sqlite.org/fts5.html.
- systemd services: freedesktop.org/software/systemd/man/systemd.service.html.
- Polkit: freedesktop.org/software/polkit/docs/latest.
- Secret Service API: specifications.freedesktop.org/secret-service/latest.
- PipeWire: pipewire.org.
- PipeWire streams: docs.pipewire.org/page_streams.html.
- Vosk offline speech recognition: alphacephei.com/vosk.
- Piper local TTS: github.com/OHF-Voice/piper1-gpl.
- openWakeWord: github.com/dscripka/openWakeWord.

91. Referências científicas fundamentais

- Vaswani et al., “Attention Is All You Need”: arxiv.org/abs/1706.03762.
- Lewis et al., “Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks”: arxiv.org/abs/2005.11401.
- Hu et al., “LoRA: Low-Rank Adaptation of Large Language Models”: arxiv.org/abs/2106.09685.
- Dettmers et al., “QLoRA: Efficient Finetuning of Quantized LLMs”: arxiv.org/abs/2305.14314.

92. Referências de APIs de providers

- OpenAI API: platform.openai.com/docs/api-reference.
- Gemini API: ai.google.dev/api.
- Anthropic API: docs.anthropic.com/en/api.
- Mistral API: docs.mistral.ai/api.
- DeepSeek API: api-docs.deepseek.com.

O adapter fixa a versão testada. Nomes de modelos e endpoints não são fundamentos do Core e podem mudar depois desta edição.

93. Decisões de engenharia que devem virar ADRs

ADR, Architecture Decision Record, registra contexto, decisão, alternativas e consequências. O repositório deve criar pelo menos:

- ADR-001: Atena é plataforma, não modelo;
- ADR-002: Core em C com providers isolados;
- ADR-003: Fase A llama-server, Fase B libllama avaliada;
- ADR-004: SQLite + FTS5 como RAG Lite;
- ADR-005: serviço systemd de usuário e Core sem root;
- ADR-006: tools tipadas e Polkit por ação;
- ADR-007: providers e models separados;
- ADR-008: memória por categorias e opt-in;
- ADR-009: pacotes .deb como unidade de integração;
- ADR-010: amd64 primeiro, ARM64 segundo, RISC-V Lab;
- ADR-011: Secure Local e bloqueio de rede;
- ADR-012: Skills sem install scripts arbitrários.

94. Próximo passo concreto

Comece pelo M0. Crie headers, schemas e provider mock. Compile com C17, execute testes e gere o primeiro pacote `atena-cli + libatena-core`. Somente depois conecte `llama-server`. Essa ordem parece menos espetacular do que abrir uma janela de chat, mas produz a fundação que permite trocar o modelo sem perder a Atena.

A arquitetura alcança seu objetivo quando a inteligência deixa de ser uma caixa-preta colocada sobre o sistema e se torna uma capacidade explicável, limitada e extensível do próprio Athenas OS.

95. Conclusão

Atena nasce da combinação entre autonomia técnica e responsabilidade. A plataforma pode operar offline, usar nuvem opcional, escolher modelos conforme hardware, pesquisar com autorização, citar documentos, falar, ouvir, ensinar e executar ferramentas. Nenhuma dessas capacidades precisa depender de um único fornecedor. Nenhuma exige entregar root ao modelo.

O desenho final preserva cinco separações: plataforma versus modelo; provider versus model; conhecimento versus memória; intenção versus efeito; usuário versus privilégio administrativo. Essas fronteiras permitem evolução sem reescrever tudo e segurança sem transformar o computador educacional em uma caixa fechada.

Para o Projeto Odisseia, isso possui sentido maior: o cyberdeck não é apenas um objeto futurista e a Atena não é apenas uma conversa convincente. Ambos são meios para que o estudante compreenda a máquina, questione decisões, teste hipóteses e construa tecnologia própria. A meta não é esconder complexidade, mas organizá-la até que possa ser aprendida.